CSC 340-02

Programming Methodology
Midterm 2 Review

Instructor: Joseph Hui

Email: jhui@sfsu.edu

Midterm/Final Announcement

- Listed in Syllabus
  - Midterm I – March 11, 2025
  - Midterm II – April 29, 2025
  - Final – May 21, 2025 (5 – 7 PM)
- All the exams in this course will be given in paper format, in the classroom and will be proctored by the instructor
- A government photo ID (student ID, driver's license or passport) is required when turning an exam
- One standard 8.5 by 11 inches allowed for notes

STL Items

The STL provides support for predefined ADTs through the use of three basic items: containers, iterators, and algorithms:

- **Containers** are objects, such as a list, vector set, etc., that hold other objects.
- **Iterators** provide a way to cycle through the contents of a container.
- **Algorithms**, such as a sorting algorithm, act on containers. STL containers are implemented as templates, similar to the ADTs implemented in this text.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

STL Containers

- Types of containers using STL
  - **Container adapters** (Restricted Interface) (e.g., stack, queue and priority queue)
    - provide efficient, restricted, position-based access to the collection
    - adapters - use other STL containers to implement their operations
  - **Sequence containers** (Ordered Collections) (e.g., array, vector, deque (double-ended queue), list and forward_list)
    - provide efficient sequential access to the collection
  - **Associative containers** (Key-Value Based) (e.g., set, map, multiset, and multimap)
    - provide efficient key-based access to the collection
- Operations common to all STL containers
  - Constructor, destructor
  - operator =
  - bool empty()
  - unit size()
- **All STL containers are part of the std:: namespace**; precede any reference to the container with std::, unless you use statement using namespace std;

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

C++ Arrays, Dynamic Arrays and STL Arrays

Feature	C++ Static Array (Built-in Array)	C++ Dynamic Array (using `new`)	C++ STL Array (`std::array`)
Size	Fixed at compile-time (cannot change)	Can change dynamically (size set at runtime)	Fixed at compile-time (cannot change)
Memory Allocation	Stack-based (or global) memory	Heap-based (using `new` and `delete`)	Stack-based (no dynamic allocation)
Bounds Checking	No bounds checking (unsafe)	No bounds checking (unsafe)	Bounds checking with `.at()` method
Size Functionality	Must calculate manually with `sizeof`	No direct way to get size without tracking manually	`.size()` member function
Utility Functions	No extra functions (basic array operations)	No built-in functions, must manage manually	Rich functionality with `.front()`, `.back()`, `.at()`, `.swap()`, etc.
Interoperability with STL	No direct support (needs pointers or manual work)	No direct support (requires pointers)	Fully interoperable with STL algorithms
Performance	Faster, no overhead	Slightly slower due to dynamic memory management	Slightly slower, but very efficient for most use cases
Copy Semantics	Shallow copy	Shallow copy	Deep copy (automatic handling of elements)

Associative Containers

- Associative Containers
  - store their entries according to **search keys**
  - two basic associative containers— **set** and **map**
    - **set** uses the entry to be stored as the key and uses a **binary tree** for its structure
    - **map** uses a **separate entry as the key**
  - both expect **unique keys**, and
  - both **store entries in ascending order** based on these keys
  - **STL multiset and STL multimap** are variations of the STL set and STL map, respectively, that **permit multiple occurrences of the search keys**

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Associative Containers

- Associative Containers
  - The STL containers **unordered_set**, **unordered_map**, **unordered_multiset**, and **unordered_multimap** are **hash-based** implementations of the corresponding ordered STL containers.
  - Since they are hash based, the unordered associative containers are **faster** for direct access of an entry based on its search key.
  - The ordered associative containers are better for **iterator-based access.**
  - The **STLmap** and STL **unordered_map** containers **permit direct access** to entries using the [ ] operator.
  - Generally, the operations of the STL map and its variations have the same behavior as those of the STL set, though the parameters are different since the STL map uses a key for each mapped value stored in the container.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

STL Algorithms

- One of the **strengths** of the STL container library is the **set of functions defined** in the header.
- These functions are designed to **operate on ranges of elements in any container with iterators**.
- This makes possible the comparison of differing types of containers, e.g., the **entries stored in a vector can be compared to those stored in a map**

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

STL Algorithms

- **Search and compare algorithms** search and compare items in a range; **they may change a value, they do not modify the structure of the containers**
- **Sequence-modification algorithms change the entries stored in a container within a range** specified by two iterators. The entries can be either
  - Replaced by a specific value using fill
  - Copied or moved to new locations using copy or move
  - Removed based on specified conditions using remove, or
  - Have operations applied to them using transform
- Sort or heap operations on a range of elements in a container.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Raw Pointers

- Allocate memory in free store by using new operator
  - Returns reference to newly created object in memory
- Store reference to object in a pointer variable
  - Use pointer variable to access object
- Copy reference to another pointer variable
  - Creates alias to same object
- Use delete operator to deallocate object's memory
  - Must also set to nullptr any pointer variables that referenced the object

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Raw Pointers

- Need to keep track number of aliases that reference an object … else results in
  - Dangling pointers
  - Memory leaks
  - Other errors (program crash, wasted memory, …)
- Languages such as Java and Python disallow direct reference to objects
  - Use reference counting to track number of aliases that reference an object
  - Known as the “reference count”
- Language can detect when object no longer has references
  - Can deallocate … known as “garbage collection”

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Smart Pointers

- C++ now supports “smart” pointers (or managed pointers; part of C++ standard in 2011)
  - Act like raw pointers
  - Also provide automatic memory management features
- When you declare a smart pointer
  - Placed on application stack
  - Smart pointer references an object ⇨ object is “managed”

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Smart Pointers

- Smart-pointer templates
  1. unique_ptr
     - no other pointer can reference same object
     - Automatically deletes the owned object when the unique_ptr goes out of scope
  2. shared_ptr
     - provides shared ownership of object
     - The object is deleted when the last shared_ptr owning it is destroyed.
  3. weak_ptr
     - reference to an object already managed by a shared pointer … does not have ownership of the object
     - It does not affect the reference count of the object, so it does not prevent it from being deleted when the last shared_ptr is destroyed

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Using Unique Pointers

- A unique pointer …
  - **Has solitary ownership** of its managed object
  - Behaves as if it maintains a **reference count of either 0 or 1** for its managed object
  - Can transfer its unique ownership of its managed object to another unique pointer using method **move**
  - **Cannot be assigned** to another unique pointer

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Unique Pointers Uses

- Unique ownership of a dynamically allocated object, **ensuring that no other part of the program can accidentally modify or delete the object**
- Automatic resource management without manually calling delete
- Transfer ownership of a resource (e.g., across function calls or between objects) without copying it
- **Avoid memory leaks and dangling pointers**, as std::unique_ptr ensures that the object is automatically destroyed when it goes out of scope

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Using Shared Pointers

- A shared pointer …
  - Provides a safe mechanism to implement shared object ownership
  - **Maintains a count** of aliases to an object
  - **Decreases or increases reference count** of managed object each time instance is created or goes out of scope or is assigned nullptr
  - **Calls destructor** of managed object when reference count reaches 0

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Shared Pointer Uses

- Shared ownership: When multiple parts of your program need to share ownership of the same object.
- Polymorphism: When you need to manage polymorphic objects, and you want to ensure that the correct destructor is called.
- Multi-threading: In multi-threaded programs where multiple threads need to access the same resource.
- Lazy initialization or caching: When you need a shared resource that should be created only when needed and destroyed automatically when no longer in use.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Using Weak Pointers

- Weak pointer only *observes* managed object
  - But does not have ownership
  - Therefore, cannot affect its lifetime
- A weak pointer …
  - References but does not own an object referenced by shared pointer
  - **Cannot affect lifetime of managed object**
  - **Does not affect reference count** of managed object
  - Has method **lock** to provide a shared-pointer version of its reference (if object has been deleted, it returns empty shared_ptr)
  - Has method **expired** to detect whether its reference object no longer exists (return true if object does not exist, else it returns false)

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Weak Pointer Uses

- Circular Dependencies: To break cycles in reference counting (circular references), where mutual ownership can lead to memory leaks.
- Observer Patterns: When you need to observe an object (e.g., a subject) without keeping it alive.
- Caching Systems: To track objects in a cache without affecting their lifetime.
- Resource Management: When you want to track resources in a system without taking ownership and without preventing their destruction.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Summary Comparison

Feature	`std::unique_ptr`	`std::shared_ptr`	`std::weak_ptr`
Ownership	Exclusive (only one `unique_ptr` can own the object)	Shared (multiple `shared_ptr` instances can own the object)	No ownership (used to observe `shared_ptr` without affecting reference count)
Reference Count	None	Uses a **reference counter**	Does not affect reference count
Copyable?	❌ No (only move is allowed)	✅ Yes (increases reference count)	✅ Yes (does not change reference count)
Moveable?	✅ Yes (ownership transfers)	✅ Yes	✅ Yes
Performance	✅ Fast (no overhead of reference counting)	❌ Slower (due to atomic reference counting)	✅ Lightweight (no reference counting)
Cyclic References?	❌ Not possible	⚠️ Possible (causes memory leaks)	✅ Used to break cyclic references

Overloaded Operators

- To overload an operator, you define an operator method whose name has the form:

  operator*symbol*

- where symbol is the operator that you want to overload
- An overloaded operator has more than one meaning
  - In class instances, must define new meaning for operators such as the equality **operator** ==
  - For lists they are equal if they have identical lengths and items

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

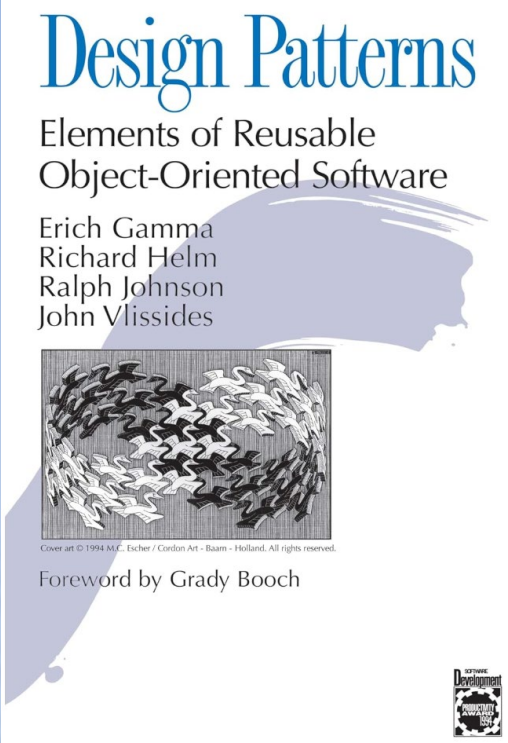
Guidelines for Overloading Operators

- Overloading an operator is the same as writing a method whose name is
  operatorsymbol
- where symbol is the operator that you want to overload.
- Overloaded operators are **typically placed in the public section of the class** declaration.
- When you decide to overload an operator, be sure that the new operation closely matches how the operator is already used. If that is not possible, consider creating a method whose name is not operatorsymbol to perform the operation.
- You can overload any C++ operator except
  . .* :: ?: sizeof
- **Commonly overloaded operators are the assignment (=), equality (= = and !=), and relational (<, <=, >, >=) operators.**
- You cannot define new operators by overloading symbols that are not already C++ operators.
- You cannot change the standard precedence of a C++ operator.
- You cannot change the number of arguments for an overloaded operator, since they represent the operator's operands.
- At least one operand of an overloaded operator must be an instance of a class.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Introducing Design Patterns

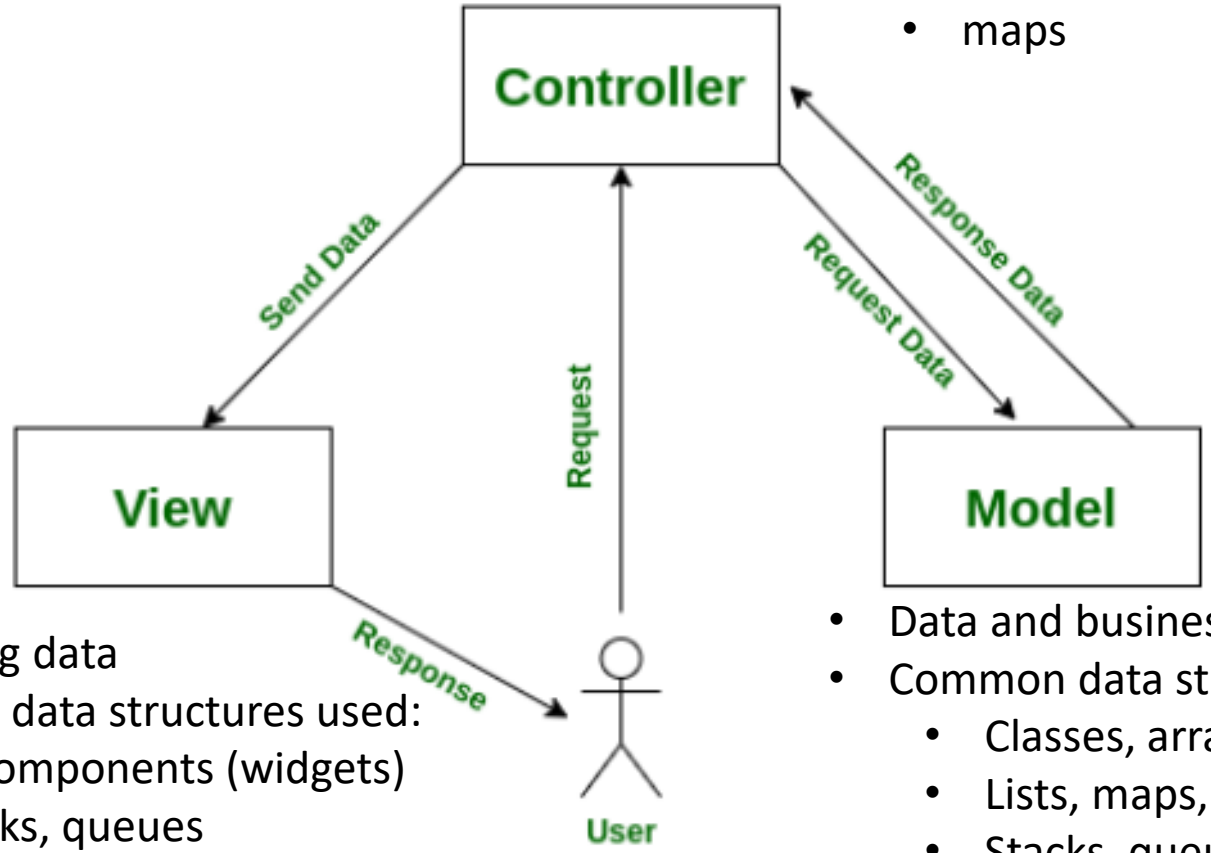
- General, reusable solutions to common problems or challenges in software design
- Programming language independent
- Describes the structure, behavior, or interaction of software components for recurring design problems in a proven and efficient manner
- Multiple Data Structures working together to solve a problem
  - Important to understand when to use specific Data Structures (advantages/disadvantages)
- 23 classic design patterns, which are categorized into three broad types:
  - Creation - deal with object creation mechanisms, trying to create objects in a way that suits the situation
  - Structural - deal with how classes and objects are composed to form larger structures
  - Behavioral - focus on communication between objects and how they interact with each other; help manage responsibilities between objects and reduce dependencies
- Interview questions (Midterm 2 and Final)!



Design Patterns - Elements of Reusable Object-Oriented Software, 1994
Gamma, Helm, Johnson, Vlissides (Gang of Four)

Behavioral - MVC

- Developed at PARC in late 70s
- Used by almost all modern UI's
- Separation of Concerns design principle
  - Distinct sections of functionality
- Some common uses:
  - Web application which updates a database
  - Mobile applications
- Highly used



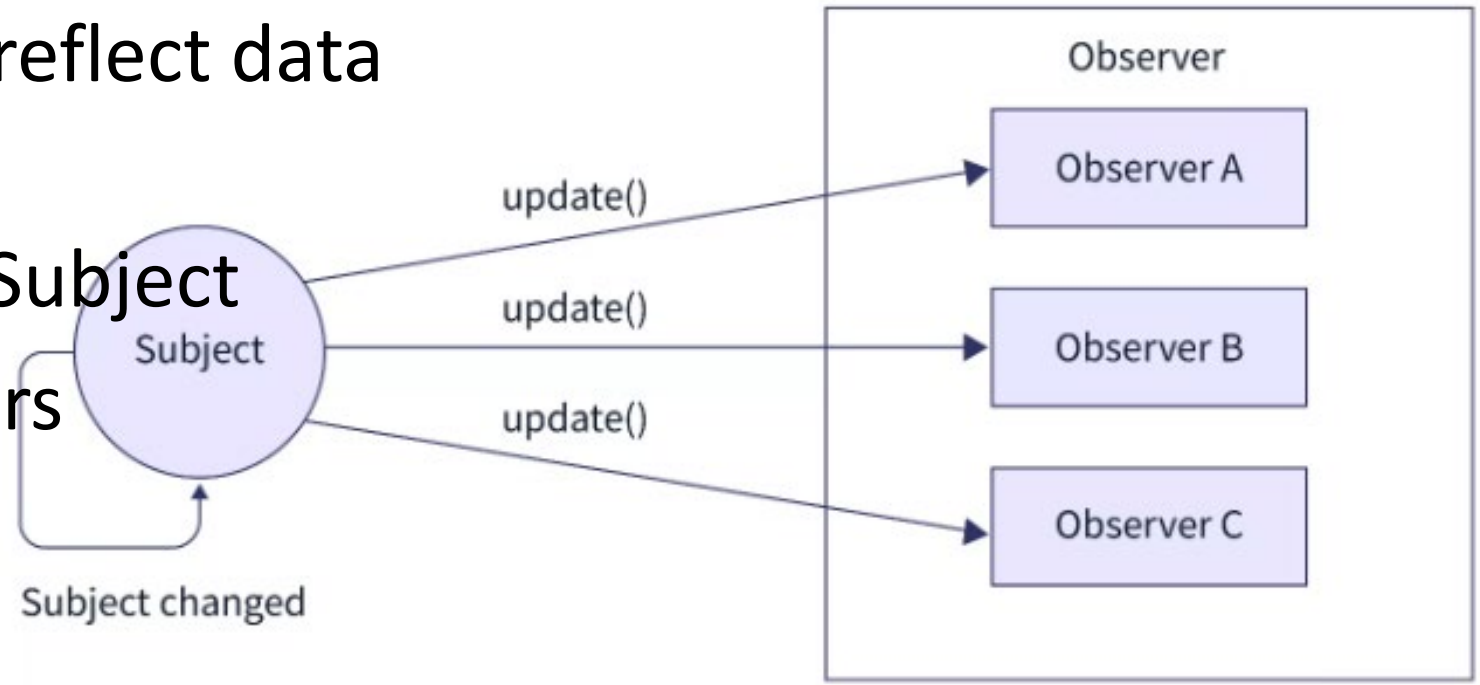
- Intermediary for model and view
- Common data structures used:
  - Classes, queues, stacks
  - maps

- Rendering data
- Common data structures used:
  - UI components (widgets)
  - Stacks, queues
  - Lists, trees

- Data and business logic
- Common data structures used:
  - Classes, arrays
  - Lists, maps, trees
  - Stacks, queues, graphs
- SQL tables

Behavioral - Observer

- Event pub/sub
- Used by modern UI's to reflect data changes
- Observers register with Subject
- Subject notifies Observers
  - Calls update()
- Some common uses:
  - News alerts
  - Stock alerts
- Highly used



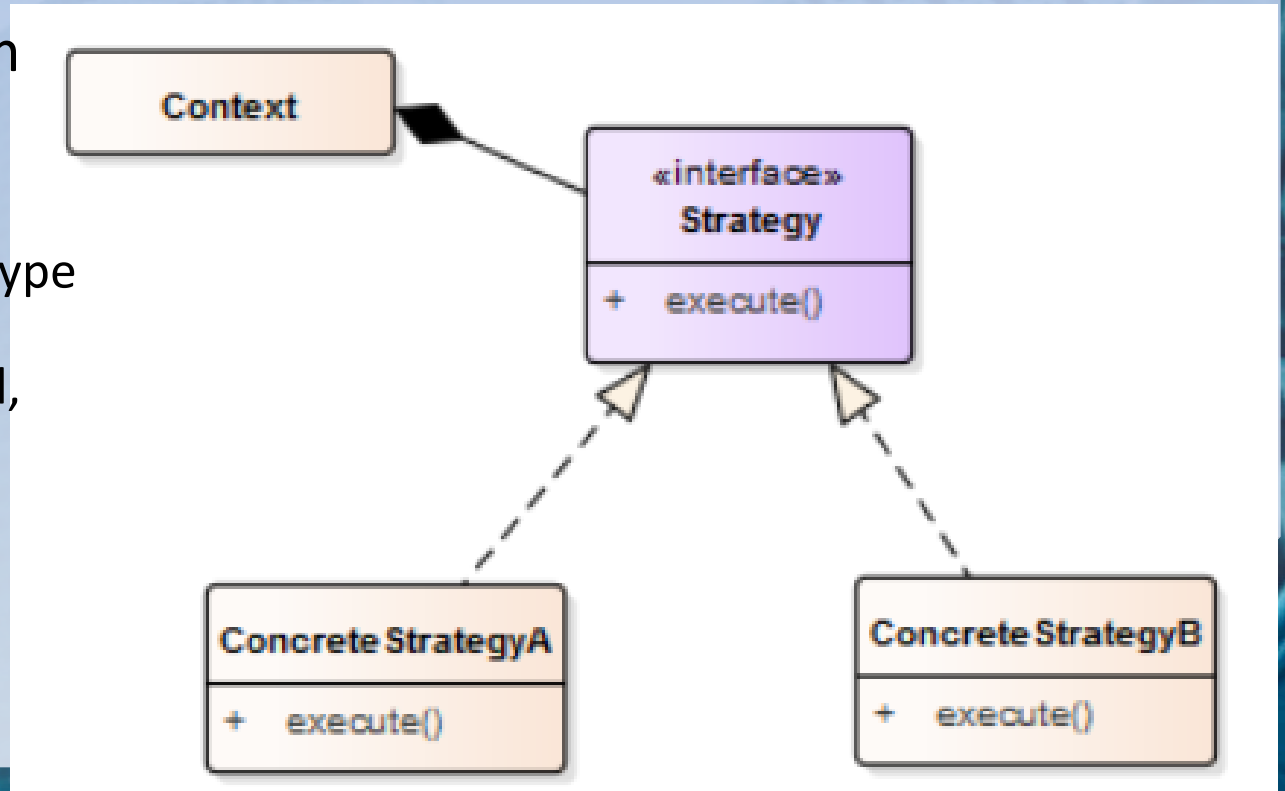
- Subject (observable) whose state changes
- Common data structures used:
  - Lists, arrays, queues
  - Set, map
  - Methods or functions (communications)

- Observer (listeners) subscribe to the subject; notified when subject state changes
- Common data structures used:
  - Classes
  - Methods or functions (event handler or callback when event occurs)

Behavioral - Strategy

- Strategy - Polymorphism at its best
- Define family of algorithms without changing the objects that use them
- Algorithm to execute is chosen at runtime
- Need different variants of an algorithm reflecting space/time trade-offs
- Some common uses:
  - Sort is dynamically selected based upon type or size of data
  - Payment systems (e.g., credit card, PayPal, bank transfer)
  - Discount calculation

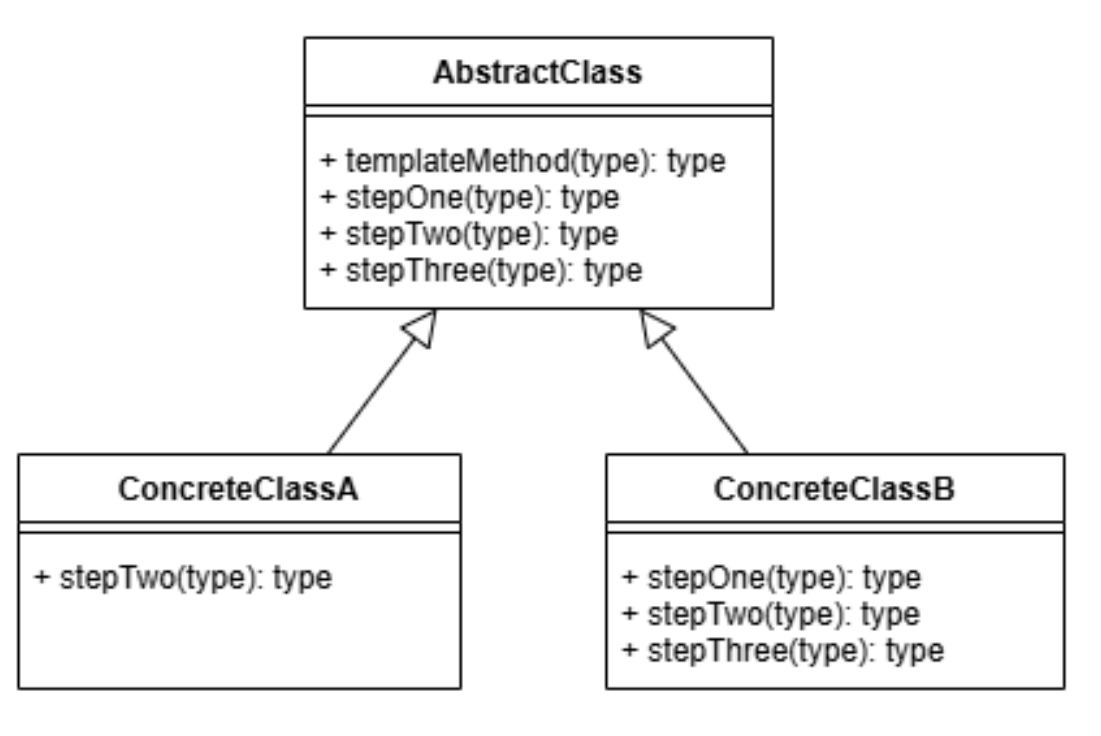
- Context (wrapper object) holds reference to strategy (can be dynamically swapped)
- Common data structures used:
  - Classes, map
  - List, array



Behavioral - Template

- Defines skeleton of an algorithm
- Some common uses:
  - Data processing (ETL – extract, transform and load) (e.g., read from database, process data, and write to CSV, JSON or XML)
  - Document processing (e.g., PDF, Word files, HTML)

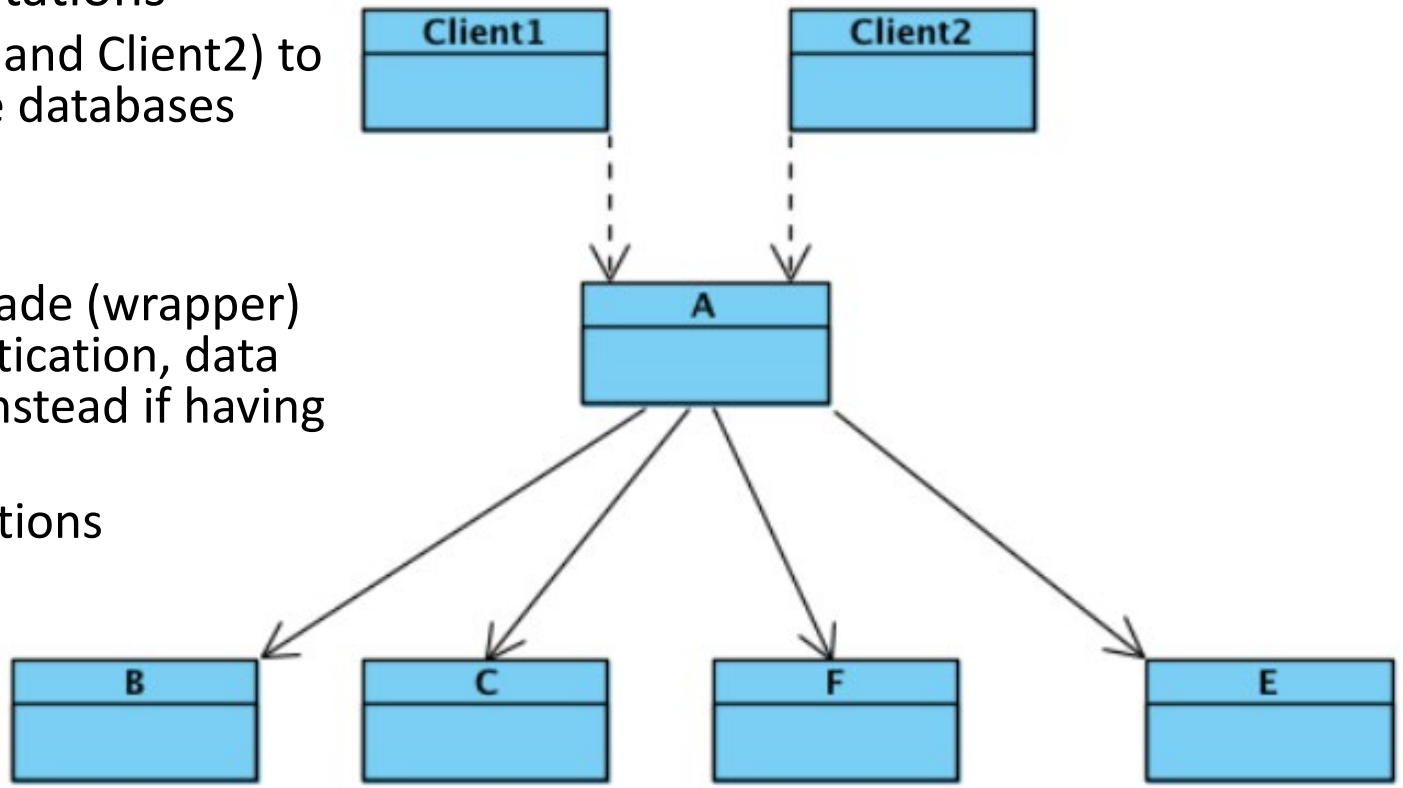
- Template (algorithm skeleton)
- Common data structures used:
  - Classes, map
  - List, array
  - Stacks, queues



Behavioral - Façade

- Provide simplified interface to complex subsystem
- Common data structures used:
  - Classes, map
  - List, array
  - Stacks, queues

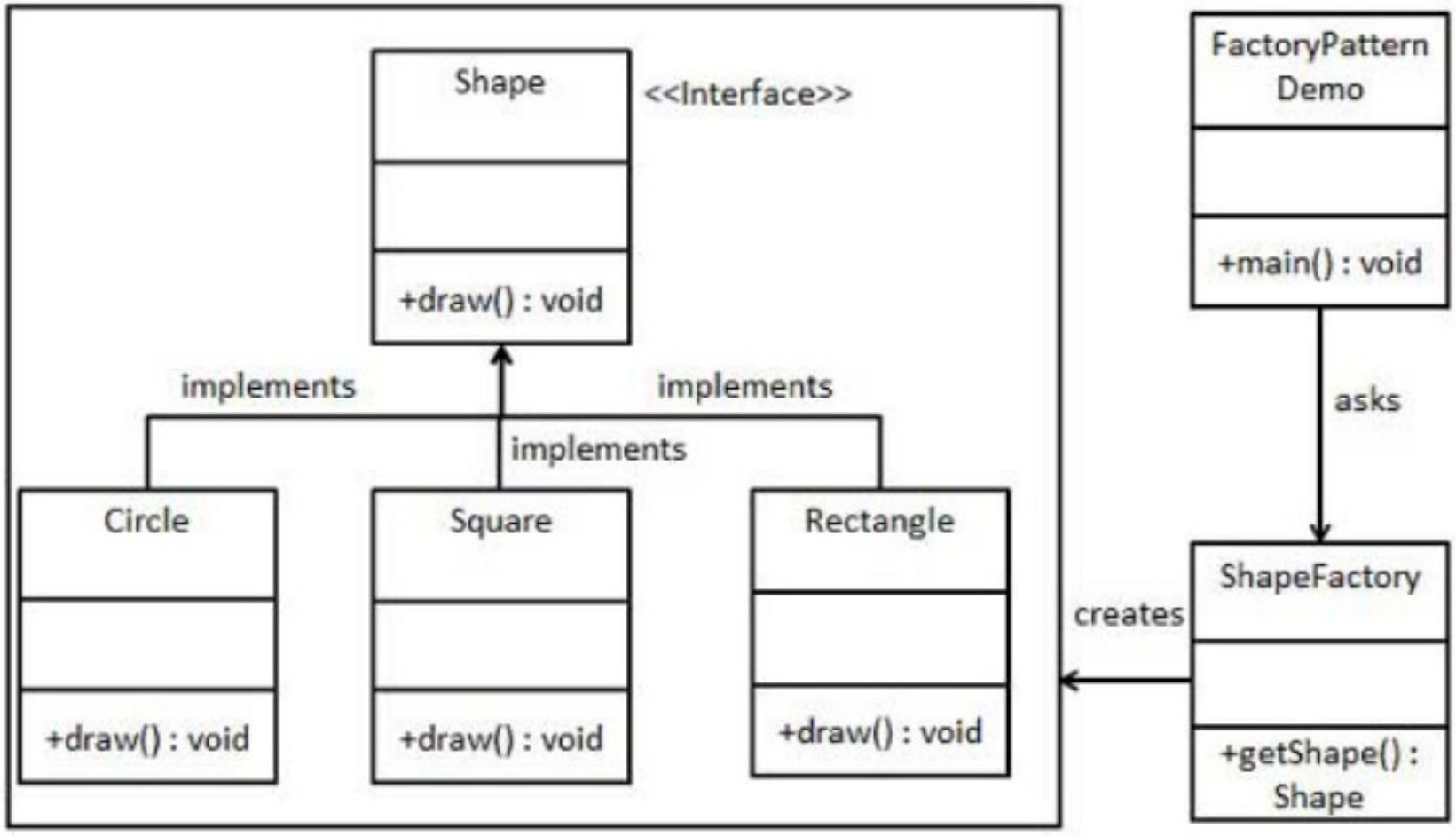
- JDBC, ODBC and DBI are good examples
- A is the JDBC, ODBC, DBI or MongoDB C++ Driver façade
- B, C, F, and E are different implementations
  - Allows different clients (i.e., Client1 and Client2) to be written once and access multiple databases
- Some common uses:
  - Database access
  - User interface design where the façade (wrapper) can be used to simplify user authentication, data retrieval and payment processing, instead if having UI interact with each service
  - Simplify subsystems in large applications



Creational - Factory

- Factory chooses which subclass to instantiate
- Allows for tracking of Object creation
- Some common uses:
  - Centralize logic for creating objects
  - Document management system that needs to create different document types (pdf, Word, etc.)
- Highly used

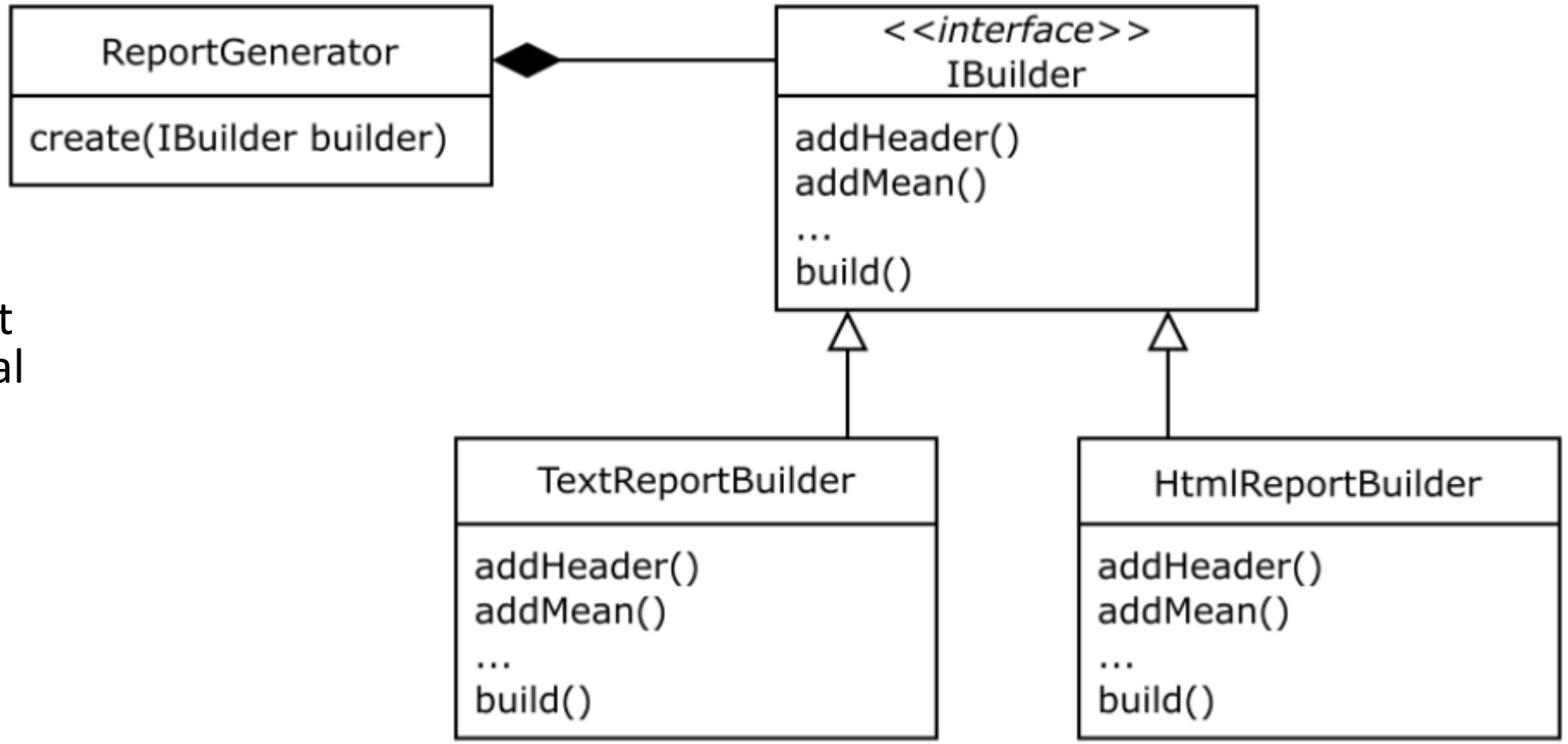
- Create objects without exposing the creation logic
- Common data structures used:
  - Classes, map
  - List, array
  - Interfaces/abstract classes



Creational - Builder

- Encapsulate complex object creation
- Delegate object creation to Builder
- Create object with many attributes or parameters
- Some common uses:
  - Create a document that has same content but different structural representations (e.g., XML, JSON)

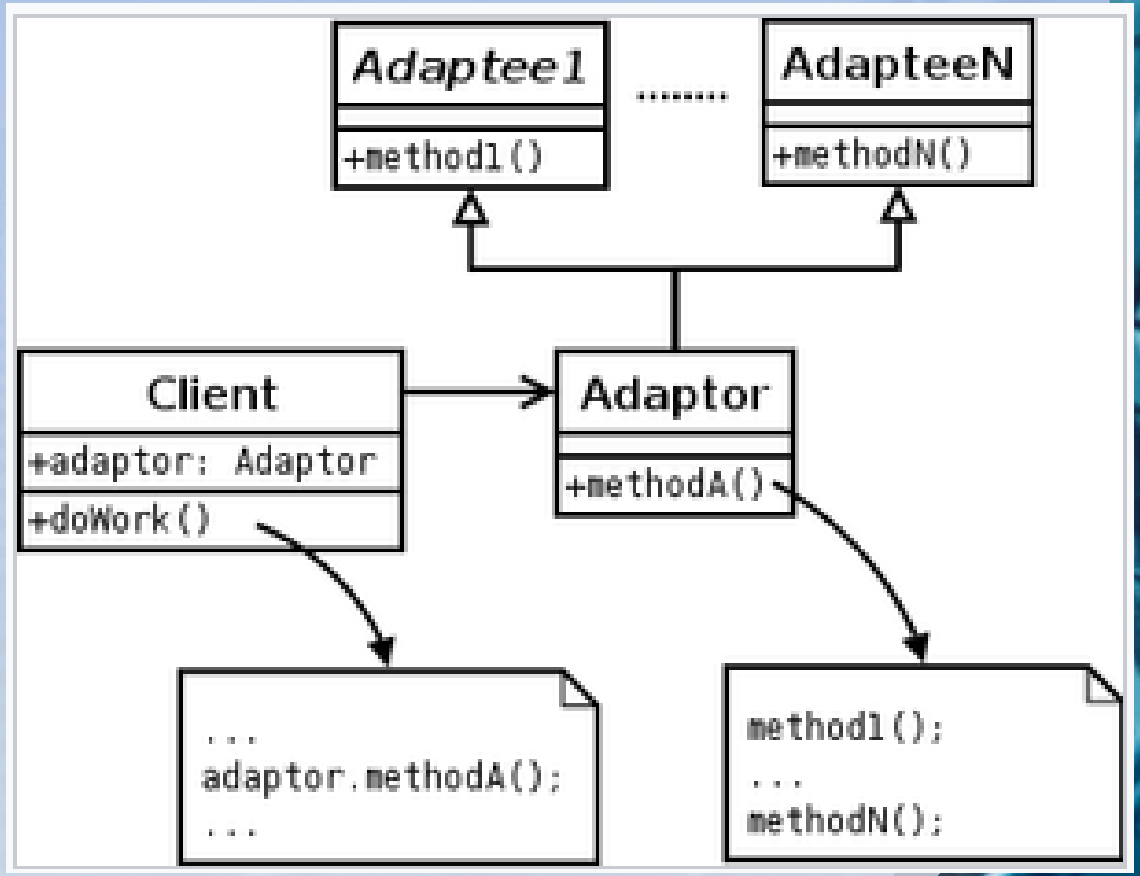
- Create different representations of an object using the same construction process
- Common data structures used:
  - Classes, map
  - List, array



Structural - Adapter

- Wrapper classes
- Converts interfaces
- Allows incompatible interfaces to work together
- Some common uses:
  - Integrate legacy systems with new code, e.g., old payment gateway to new user interface
  - Adapting a json API to an XML API
  - Converting data formats (e.g., XML to JSON or vice versa)

- Enable an interface of a class to be used as another interface; allows incompatible interfaces to work together
- Common data structures used:
  - Classes, map
  - List, array
  - Queues, stacks, buffers

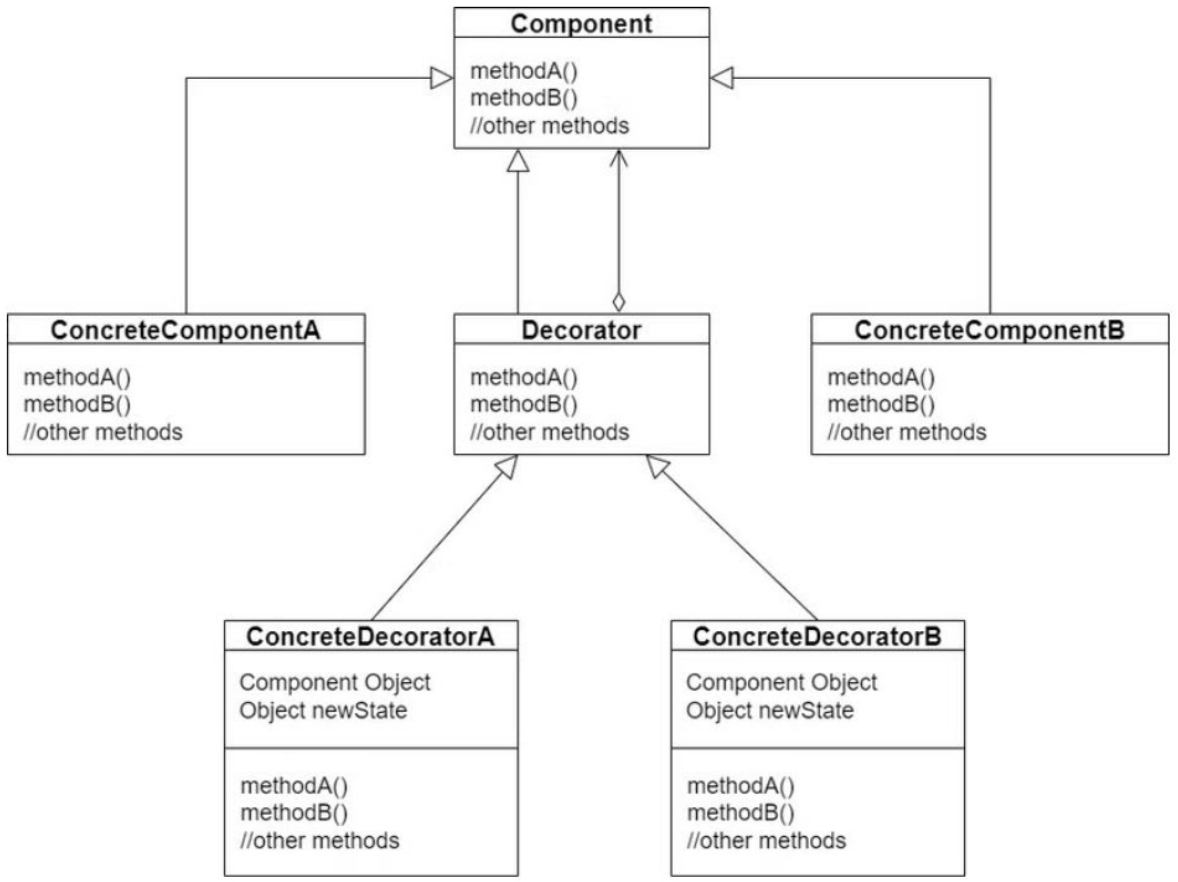


Structural - Decorator

- Allows new functionality to be added to an object dynamically
- Open/Closed design principle
  - Open for extension
  - Close for modification

- Extension Options
  - Inheritance
    - Is a relationship
  - Composition
    - Has a relationship
    - Note that decorators have Component embedded
- Some common uses:
  - Add borders, shadows or other visual effects in a graphical application
  - Add logging, caching or transaction management, but don't want to modify original class implementation

- Create new functionality to an object dynamically, without modifying its structure; it extends functionality of a class
- Common data structures used:
  - Classes, map
  - List, sets
  - Queues, stacks

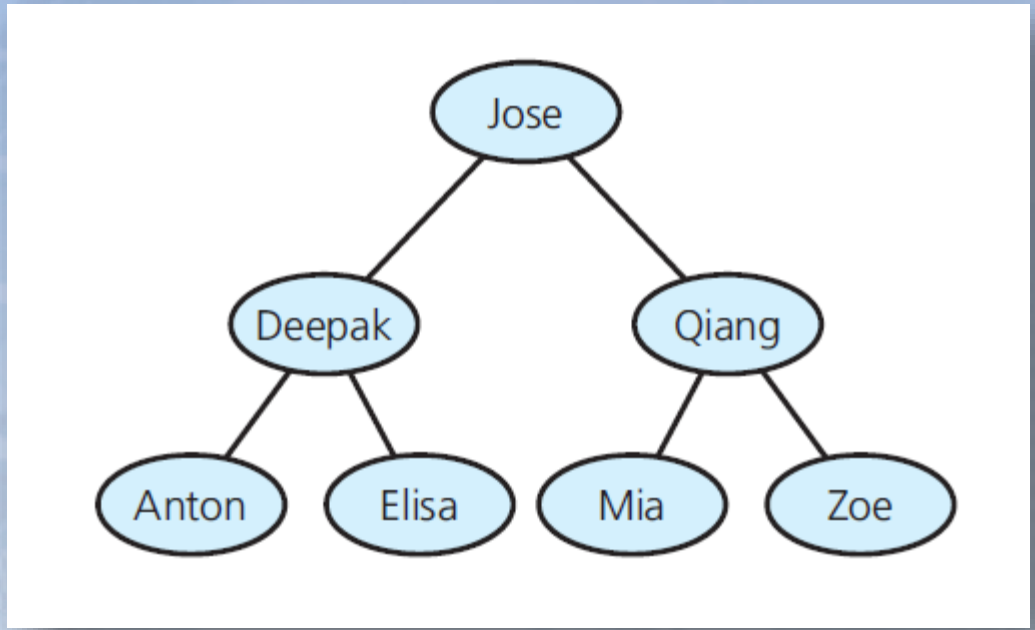


Kinds of Trees

- General tree
  - Set *T* of one or more nodes (not empty)
  - *T* is partitioned into disjoint subsets
- An n-ary tree
  - A single node r, the root
  - n possibly empty sets that are n-ary subtrees of r
  - Each node can have no more than n children
- Binary tree
  - Set of *T* nodes – either empty or partitioned into disjoint subsets
  - Single node *r*, the root
  - Two (possibly empty) sets – left and right subtrees

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Kinds of Trees



A binary search tree is a binary tree that is, sorted according to the values in its nodes. For each node n, a binary search tree satisfies the following three properties:

1. n ’s value is greater than all values in its left subtree TL
2. n ’s value is less than all values in its right subtree TR
3. Both TL and TR are binary search trees.

FIGURE 15-4 A binary search tree of names

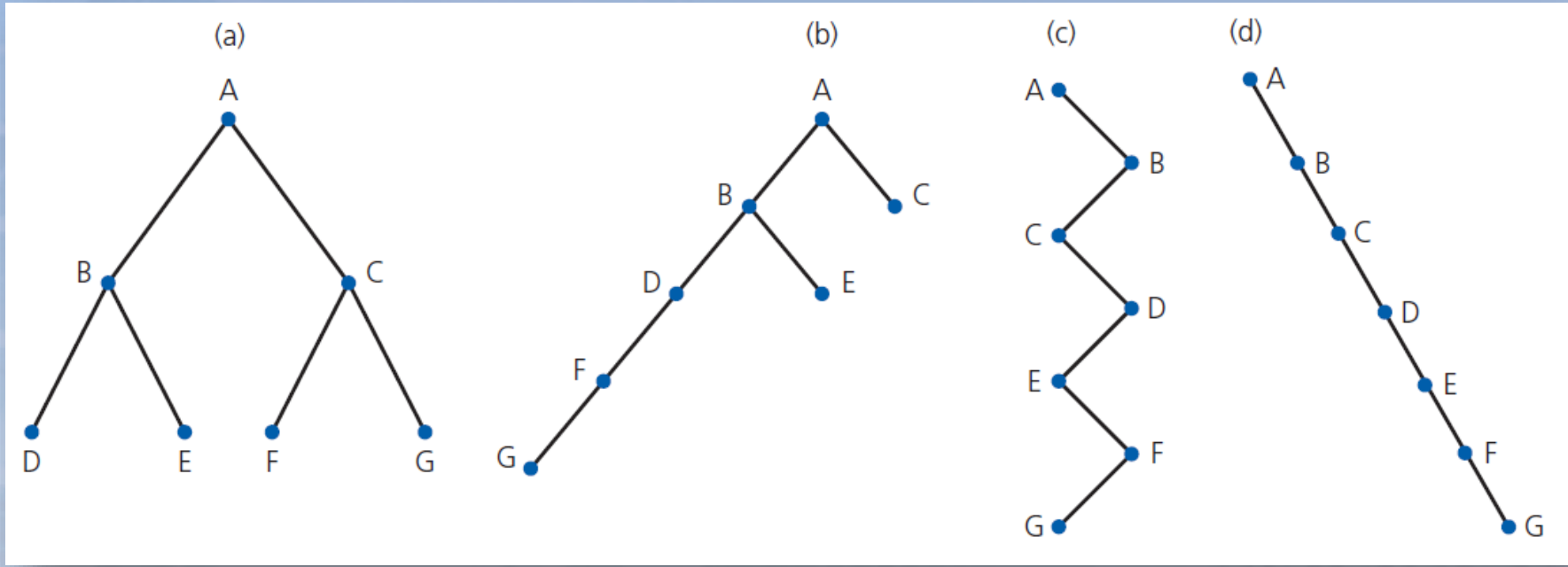
© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Height of Trees

- Level of a node, *n*
  - If *n* is root, level 1
  - If *n* not the root, level is 1 greater than level of its parent
- Height of a tree
  - Number of nodes on longest path from root to a leaf
  - *T* empty, height 0
  - *T* not empty, height equal to max level of nodes

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Height of Trees

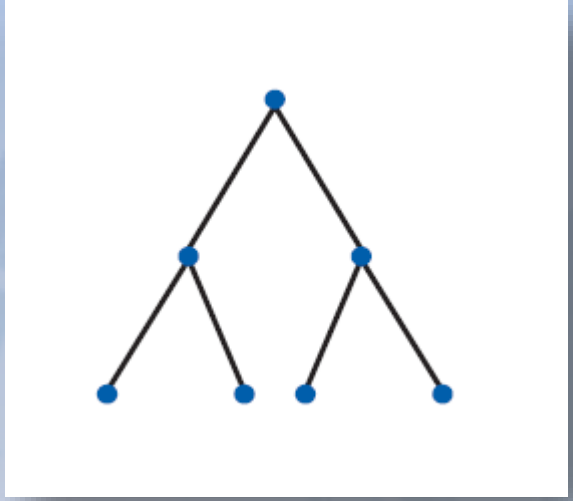


The heights are:3, 5, 7, and 7 respectively.

FIGURE 15-5 Binary trees with the same nodes but different heights

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Full, Complete, and Balanced Binary Trees

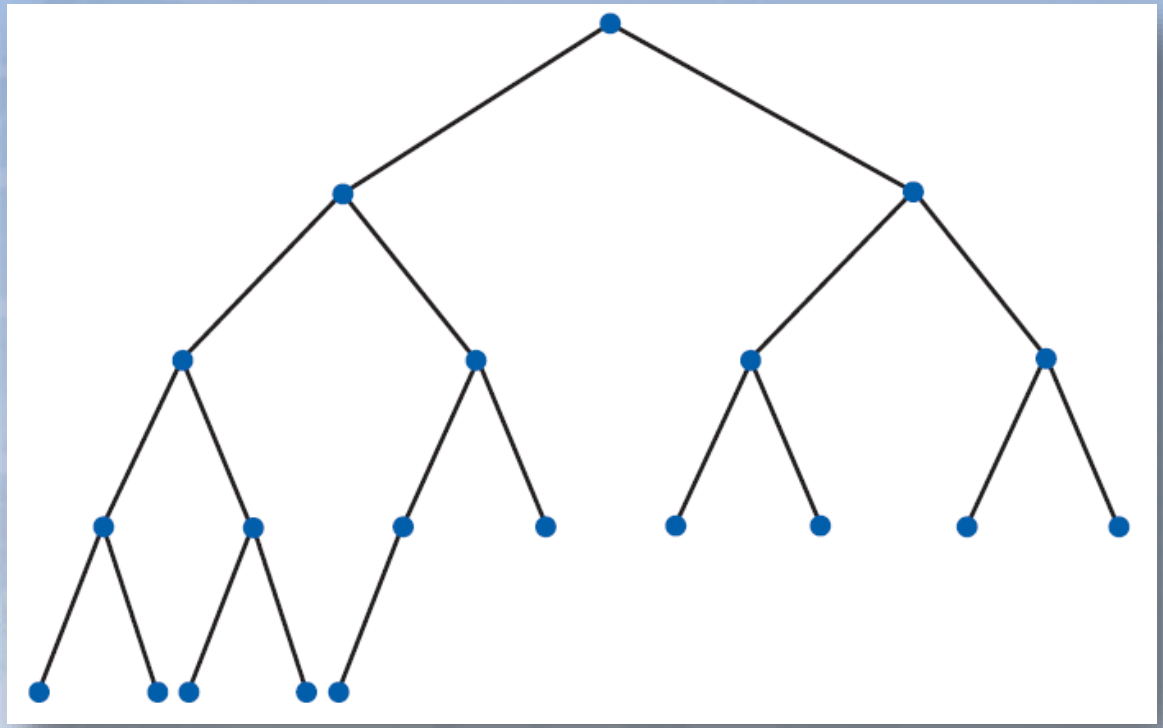


In a **full** binary tree of height h, all nodes that are at a level less than h have two children each.

FIGURE 15-6 A **full** binary tree of height 3

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Full, Complete, and Balanced Binary Trees



A **complete** binary tree of height h is a binary tree that is full down to level h – 1, with level h filled in from left to right:

1. All nodes at level h – 2 and above have two children each
2. When a node at level h – 1 has children, all nodes to its left at the same level have two children each
3. When a node at level h – 1 has one child, it is a left child

FIGURE 15-7 A **complete** binary tree

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Full, Complete, and Balanced Binary Trees

- Full binary trees are complete
  - Binary tree of height h with no missing nodes.
  - All leaves are at level h
  - All other nodes each have two children.
- Complete binary trees are balanced
  - Binary tree of height h that is full to level h – 1 and has level h filled in from left to right.
- Balanced binary tree
  - Binary tree in which the left and right subtrees of any node have heights that differ by at most 1

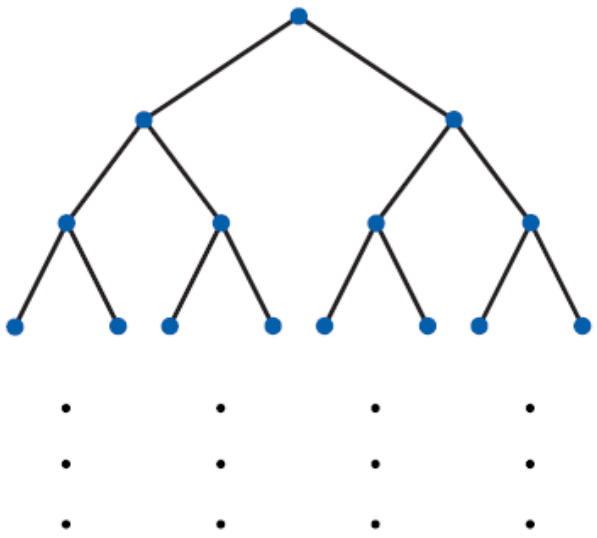
© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Maximum and Minimum Heights of a Binary Tree

- Binary tree with *n* nodes
  - Max height is *n*
- To minimize height of binary tree of *n* nodes
  - Fill each level of tree as completely as possible
  - A complete tree meets this requirement

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Maximum and Minimum Heights of a Binary Tree



Level	Number of nodes at this level	Total number of nodes at this level and all previous levels
1	$1 = 2^0$	$1 = 2^1 - 1$
2	$2 = 2^1$	$3 = 2^2 - 1$
3	$4 = 2^2$	$7 = 2^3 - 1$
4	$8 = 2^3$	$15 = 2^4 - 1$
⋮	⋮	⋮
h	2^{h-1}	$2^h - 1$

Maximum # of nodes for a binary tree

FIGURE 15-9 Counting the nodes in a full binary tree of height *h*

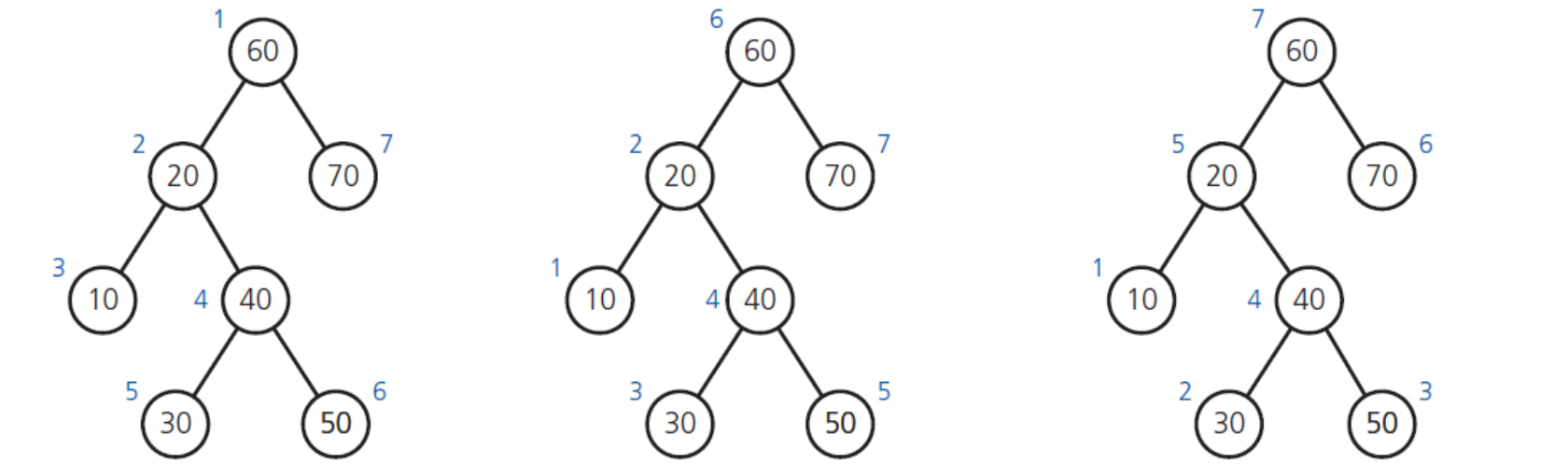
© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Traversals of a Binary Tree

- Options for when to visit the root
  - Preorder: before it traverses both subtrees
  - Inorder: after it traverses left subtree, before it traverses right subtree
  - Postorder: after it traverses both subtrees
- Note traversal is O(*n*)

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Traversals of a Binary Tree



(a) Preorder: 60, 20, 10, 40, 30, 50, 70 (b) Inorder: 10, 20, 30, 40, 50, 60, 70 (c) Postorder: 10, 30, 50, 40, 20, 70, 60

(Numbers beside nodes indicate traversal order.)

FIGURE 15-11 Three traversals of a binary tree

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Traversals of a Binary Tree

```
// Traverses the given binary tree in preorder.
// Assumes that "visit a node" means to process the node's data item.
preorder(binTree: BinaryTree): void
{
   if (binTree is not empty)
   {
      Visit the root of binTree
      preorder(Left subtree of binTree's root)
      preorder(Right subtree of binTree's root)
   }
}
```

Preorder traversal algorithm

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Traversals of a Binary Tree

```
// Traverses the given binary tree in inorder.
// Assumes that "visit a node" means to process the node's data item.
inorder(binTree: BinaryTree): void
{
   if (binTree is not empty)
   {
      inorder(Left subtree of binTree's root)
      Visit the root of binTree
      inorder(Right subtree of binTree's root)
   }

}
```

Inorder traversal algorithm

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Traversals of a Binary Tree

```
// Traverses the given binary tree in postorder.
// Assumes that "visit a node" means to process the node's data item.
postorder(binTree: BinaryTree): void
{   if (binTree is not empty)
    {
        postorder(Left subtree of binTree's root)
        postorder(Right subtree of binTree's root)
        Visit the root of binTree
    }
}
```

Postorder traversal algorithm

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Searching a Binary Search Tree

Search algorithm for a binary search tree

```
// Searches the binary search tree for a given target value.
search(bstTree: BinarySearchTree, target: ItemType)
{
   if (bstTree is empty)
      The desired item is not found
   else if (target == data item in the root of bstTree)
      The desired item is found
   else if (target < data item in the root of bstTree)
      search(Left subtree of bstTree, target)
   else
      search(Right subtree of bstTree, target)
}
```

- This algorithm will work regardless of the shape of a binary search tree.
- The worse case of a binary search tree is O(n), when it is not balanced.
- The average time complexity for a binary search tree is O(log n)
- The best case of a binary search tree is O(1).

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Traversals of a Binary Search Tree

```
// Traverses the given binary tree in inorder.
// Assumes that "visit a node" means to process the node's data item.
inorder(binTree: BinaryTree): void
{
   if (binTree is not empty)
   {
      inorder(Left subtree of binTree's root)
      Visit the root of binTree
      inorder(Right subtree of binTree's root)
   }

}
```

Inorder traversal of a binary search tree visits tree's nodes in sorted search-key order

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Basic Sorting Algorithms

- The Selection Sort
- The Bubble Sort
- The Insertion Sort

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Selection Sort

- Select the largest item
  - Place in last position
- Select the next largest item
  - Place in 2nd to last position
- Repeat for remaining unsorted items
  - Requires several passes over the data

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Selection Sort

- Analysis
  - Selection sort is $O(n^2)$
  - Appropriate for small *n*
  - $O(n^2)$ grows rapidly
- Could be a good choice when
  - Data moves are costly,
  - But comparisons are not
- Pros
  - Easy to understand
  - Minimal additional space required
- Cons
  - $O(n^2)$ – slow
  - Does not preserve relative order of items with equal keys (not stable)
  - Potentially suitable for low volume only

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Bubble Sort

- Compares adjacent items
  - Exchanges them if out of order
  - Requires several passes over the data
- When ordering successive pairs
  - Largest item bubbles to end of the array

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Bubble Sort

- Analysis
  - Worst case $O(n^2)$
  - Best case (array already in order) is $O(n)$
- Pros
  - Easy to understand
  - No additional space required
  - Preserves relative order of items with equal keys (stable)
- Cons
  - $O(n^2)$ – slow
  - Potentially suitable for low volume only

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Insertion Sort

- Take each item from unsorted region
  - Insert it into correct order in sorted region

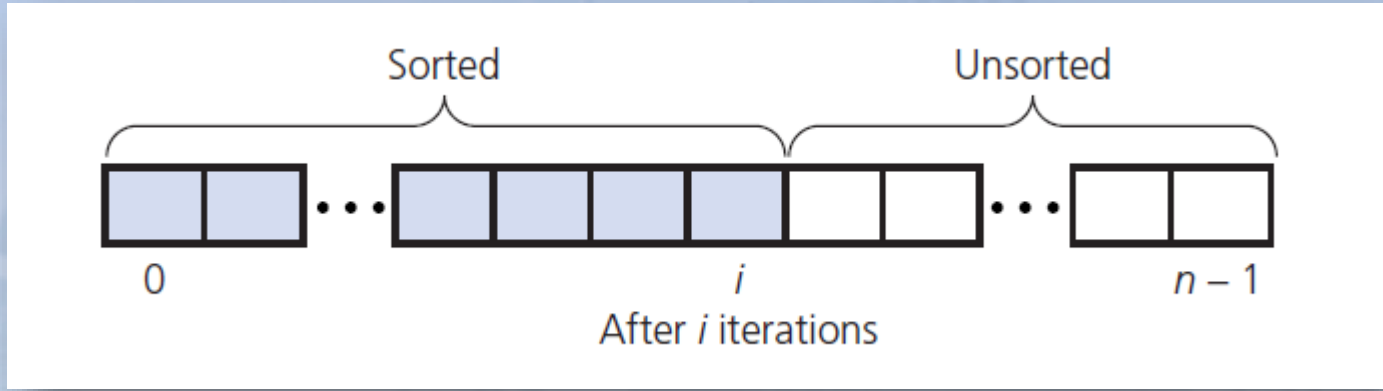


FIGURE 11-3 An insertion sort partitions the array into two regions

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Insertion Sort

- Analysis
  - Worst case $O(n^2)$
  - Best case (array already in order) is $O(n)$
- Unsuitable for large arrays
  - Unless already sorted
- Pros
  - Easy to understand
  - No additional space required
  - Preserves relative order of items with equal keys (stable)
- Cons
  - $O(n^2)$ – slow
  - Potentially suitable for low volume only

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Faster Sorting Algorithms

- The Merge Sort
- The Quick Sort
- The Radix Sort

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Merge Sort

- Divide and conquer sorting algorithm
- Halve the array, sort its halves, and then merge the halves

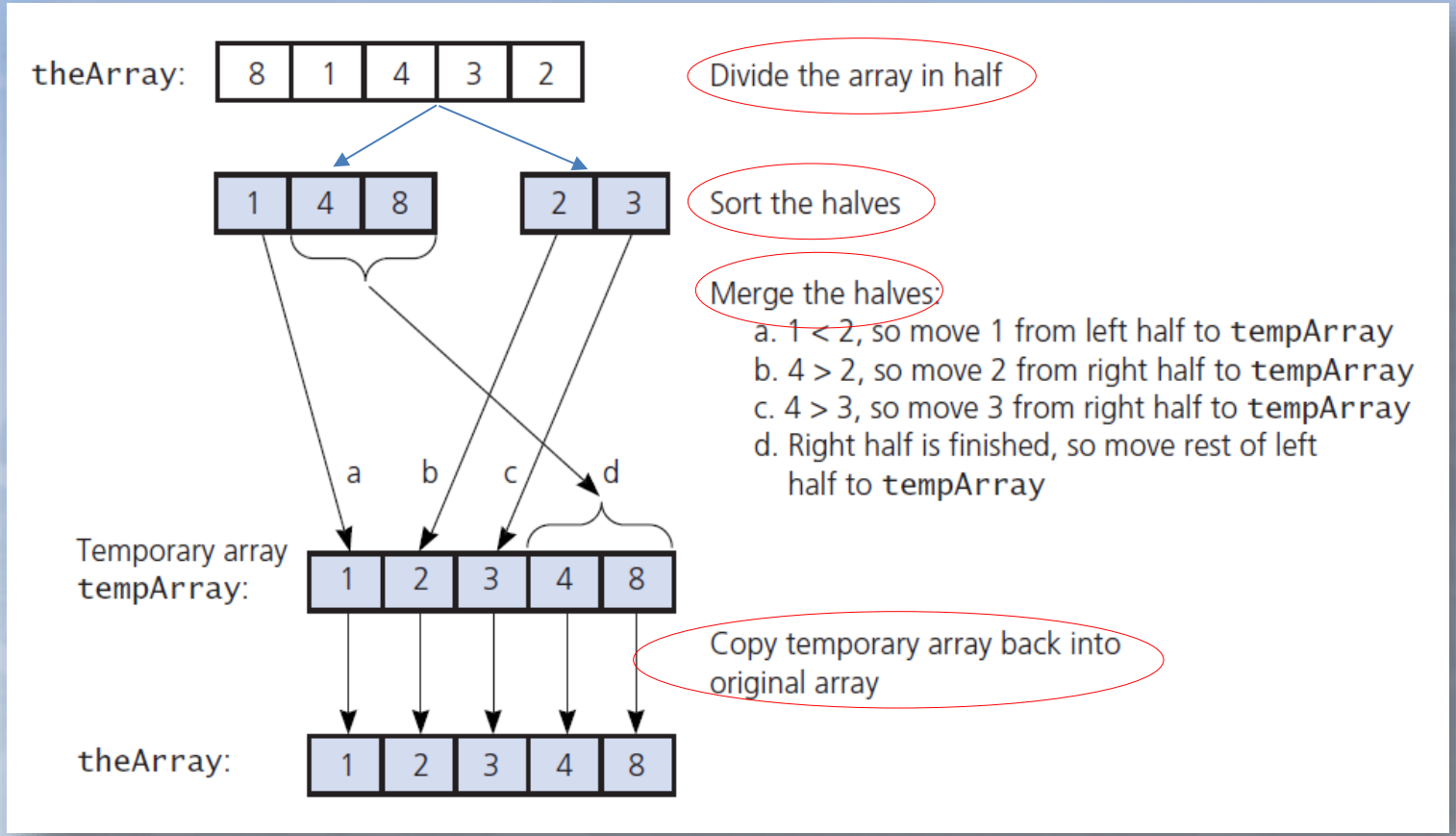


FIGURE 11-5 A merge sort with an auxiliary temporary array

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Merge Sort

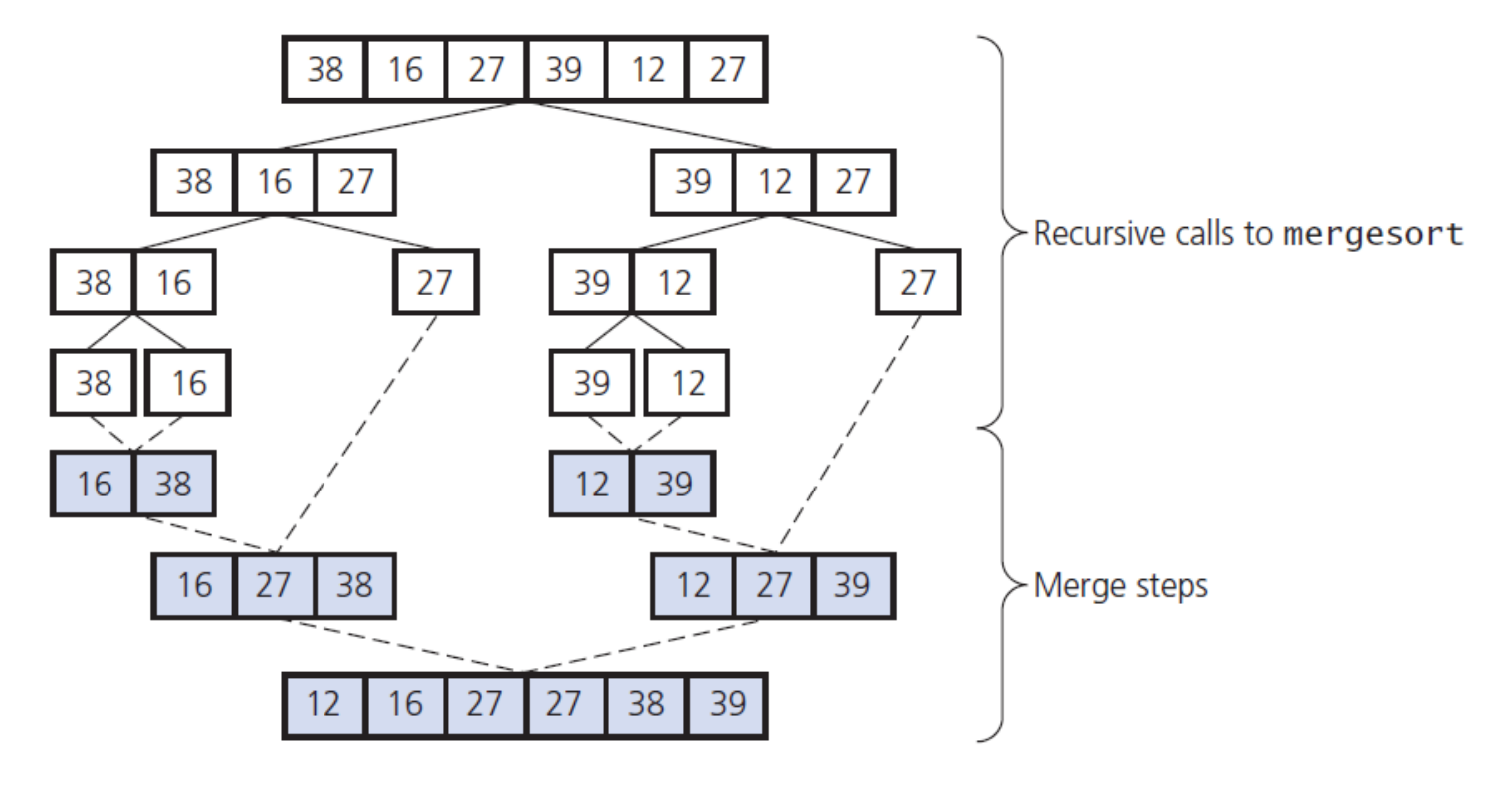


FIGURE 11-6 A merge sort of an array of six integers

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Merge Sort

- Analysis
  - O(n log n)
  - Worse case O(n log n)

- Pros
  - Performs well even on large number of items
  - Simple to implement
  - Preserves relative order of items with equal keys (stable)
- Cons
  - Requires additional memory O(n)
  - Requires copying of entries

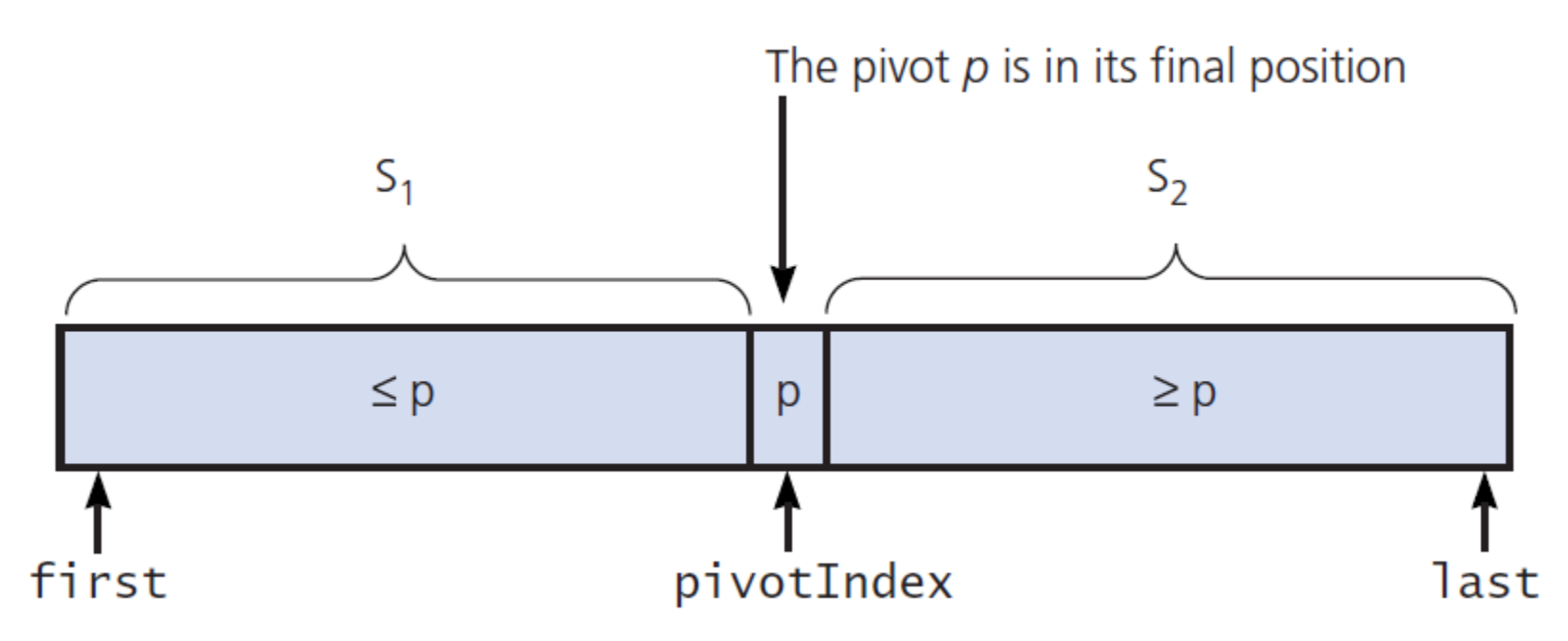
© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Quick Sort

- Another divide-and-conquer algorithm
  - Choose a pivot
  - Partition an array into items that are
    - Less than or equal to the pivot and
    - Those that are greater than or equal to the pivot
  - Recursively apply the same process to the partitioned sub-arrays
  - Recursion stops when there is one element remaining in the sub-array

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Quick Sort



Quick sort algorithm

1. Choose a pivot
2. Partition the array
3. Recursively call

FIGURE 11-9 A partition about a pivot

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Quick Sort

- Pivot selection goal
  - Subarrays each have the same or nearly the same number of entries
- Pivot selection strategy
  - Last entry
  - First entry
  - Middle entry
  - Median-of-three

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Quick Sort

- Analysis
  - Partitioning is an O(n) task
  - There are either $\log_2 n$ or 1 + $\log_2 n$ levels of recursive calls to quickSort.
- We conclude
  - Worst case O(n^2)
  - Average case O(n log n)
- Pros
  - Divide and conquer algorithm that is efficient on large number of items
  - No additional memory required
- Cons
  - Worst case O(n^2)
  - Does not preserve relative order of items with equal keys (not stable)

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Radix Sort

- Different from other sorts
  - Does not compare entries in an array
- Begins by organizing data (say strings) according to least significant letters
  - Then combine the groups
- Next form groups using next least significant letter

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Radix Sort

0123, 2154, 0222, 0004, 0283, 1560, 1061, 2150	Original integers
(1560, 2150) (1061) (0222) (0123, 0283) (2154, 0004)	Grouped by fourth digit
1560, 2150, 1061, 0222, 0123, 0283, 2154, 0004	Combined
(0004) (0222, 0123) (2150, 2154) (1560, 1061) (0283)	Grouped by third digit
0004, 0222, 0123, 2150, 2154, 1560, 1061, 0283	Combined
(0004, 1061) (0123, 2150, 2154) (0222, 0283) (1560)	Grouped by second digit
0004, 1061, 0123, 2150, 2154, 0222, 0283, 1560	Combined
(0004, 0123, 0222, 0283) (1061, 1560) (2150, 2154)	Grouped by first digit
0004, 0123, 0222, 0283, 1061, 1560, 2150, 2154	Combined (sorted)

FIGURE 11-13 A radix sort of eight integers

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Radix Sort

- Analysis
  - Requires *n* moves each time it forms groups
  - *n* moves to combine again into one group
  - Performs these $2 \times n$ moves *d* times
  - Thus requires $2 \times n \times d$ moves
  - More appropriate for chain of linked lists than for an array
- Radix sort is of order O(*n*)
- Pros
  - Radix sort is of order O(n)
  - Often faster than merge sort or quick sort for large number of items
  - Preserves relative order of items with equal keys (stable)
- Cons
  - Time complexity grows linearly with the number of digits
  - Requires same size keys, with standard way of breaking the keys into pieces

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

A Comparison of Sorting Algorithms

	Worst case	Average case
Selection sort	n^2	n^2
Bubble sort	n^2	n^2
Insertion sort	n^2	n^2
Merge sort	$n \times \log n$	$n \times \log n$
Quick sort	n^2	$n \times \log n$
Radix sort	n	n
Tree sort	n^2	$n \times \log n$
Heap sort	$n \times \log n$	$n \times \log n$

FIGURE 11-14 Approximate growth rates of time required for eight sorting algorithms

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Implementation

```
template<class ItemType>
std::shared_ptr<BinaryNode<ItemType>> BinaryNodeTree<ItemType>::copyTree(
     const std::shared_ptr<BinaryNode<ItemType>> oldTreeRootPtr) const
{
   std::shared_ptr<BinaryNode<ItemType>> newTreePtr;

   // Copy tree nodes during a preorder traversal
   if (oldTreeRootPtr != nullptr)
   {
      // Copy node
      newTreePtr = std::make_shared<BinaryNode<ItemType>>(oldTreeRootPtr->getItem(),
                                                          nullptr, nullptr);
      newTreePtr->setLeftChildPtr(copyTree(oldTreeRootPtr->getLeftChildPtr()));
      newTreePtr->setRightChildPtr(copyTree(oldTreeRootPtr->getRightChildPtr()));
   }  // end if
   // Else tree is empty (newTreePtr is nullptr)

   return newTreePtr;
}  // end copyTree
```

To copy a tree, traverse it in **preorder** and insert each item visited into a new node

Protected method copyTree called by copy constructor

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Implementation

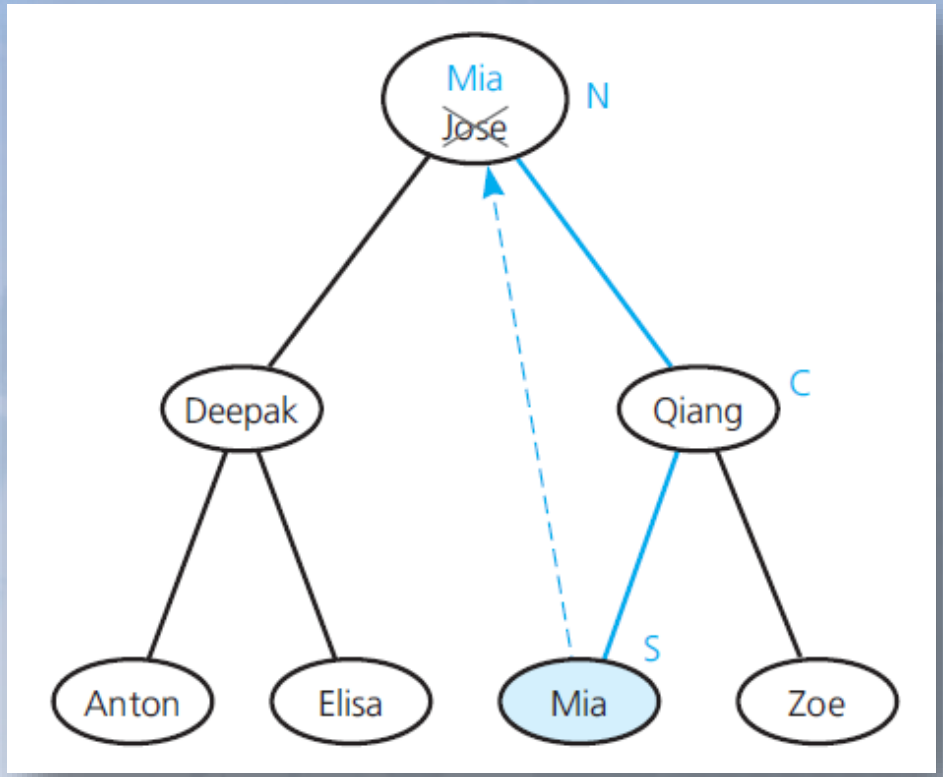
To delete a tree, traverse it in **postorder** and delete

```
template<class ItemType>
void BinaryNodeTree<ItemType>::
     destroyTree(std::shared_ptr<BinaryNode<ItemType>> subTreePtr)
{
   if (subTreePtr != nullptr)
   {
      destroyTree(subTreePtr->getLeftChildPtr());
      destroyTree(subTreePtr->getRightChildPtr());
      subTreePtr.reset(); // Decrement reference count to node
   }  // end if
}  // end destroyTree
```

destroyTree used by destructor which simply calls this method

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Link-Based Implementation of the ADT Binary Search Tree

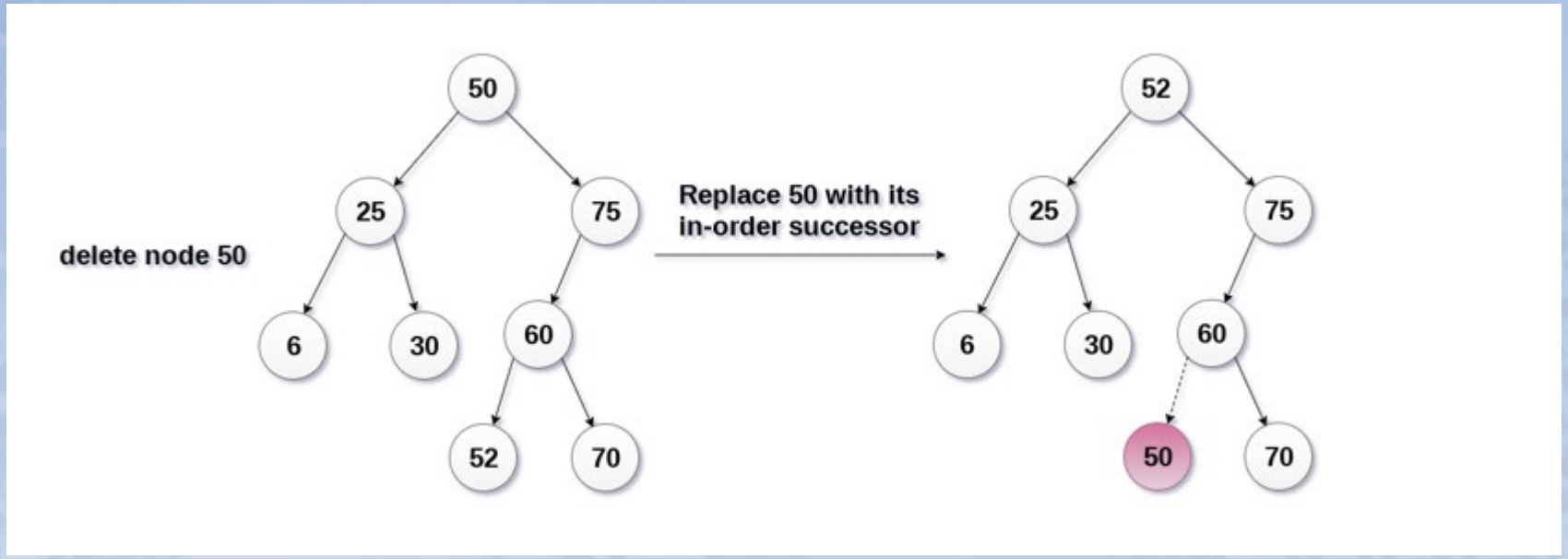


- inorder successor of a data item is the leftmost node of N's right subtree.

FIGURE 16-13 Replacing the data item in node *N* with its **inorder successor**

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Link-Based Implementation of the ADT Binary Search Tree



https://www.javatpoint.com/deletion-in-binary-search-tree

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Saving a Binary Search Tree in a File

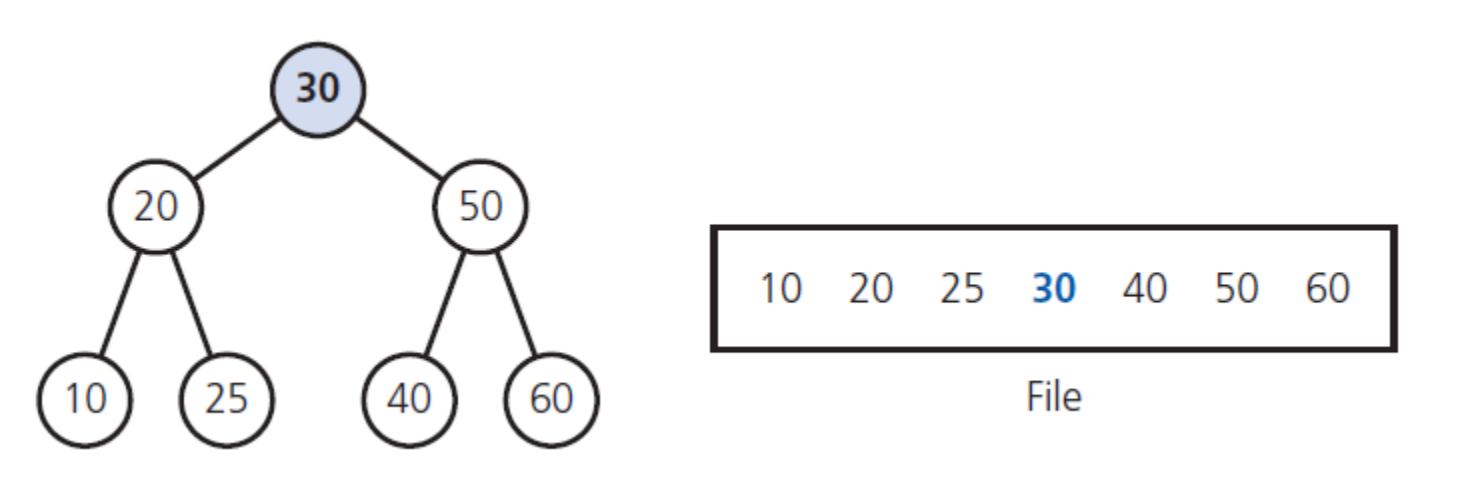


FIGURE 16-16 A full tree saved in a file by using inorder traversal

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Tree Sort

```
// Sorts the integers in an array into ascending order.
treeSort(anArray: array, n: integer)
{
    Add anArray's entries to a binary search tree bst
    Traverse bst in inorder. As you visit bst's nodes, copy their data items into successive
     locations of anArray
}
```

- Tree sort:
  - Insertion into BST is O(log n) in the average case, and O(n) in the worse case
  - Thus, insertion into BST is O(n × log n) in the average case, and $O(n^2)$ in the worst case
  - Traversal involves one copy: O(n)
  - Thus, tree sort is O(n × log n) in the average case, and $O(n^2)$ in the worst case

Tree sort uses a binary search tree.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The ADT Heap

- A heap is a complete binary tree that either is
  - Empty or …
  - Whose root contains a value ≥ each of its children and has heaps as its subtrees
- It is a special binary tree … different in that
  - It is ordered in a weaker sense
  - it will always be a complete binary tree

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The ADT Heap

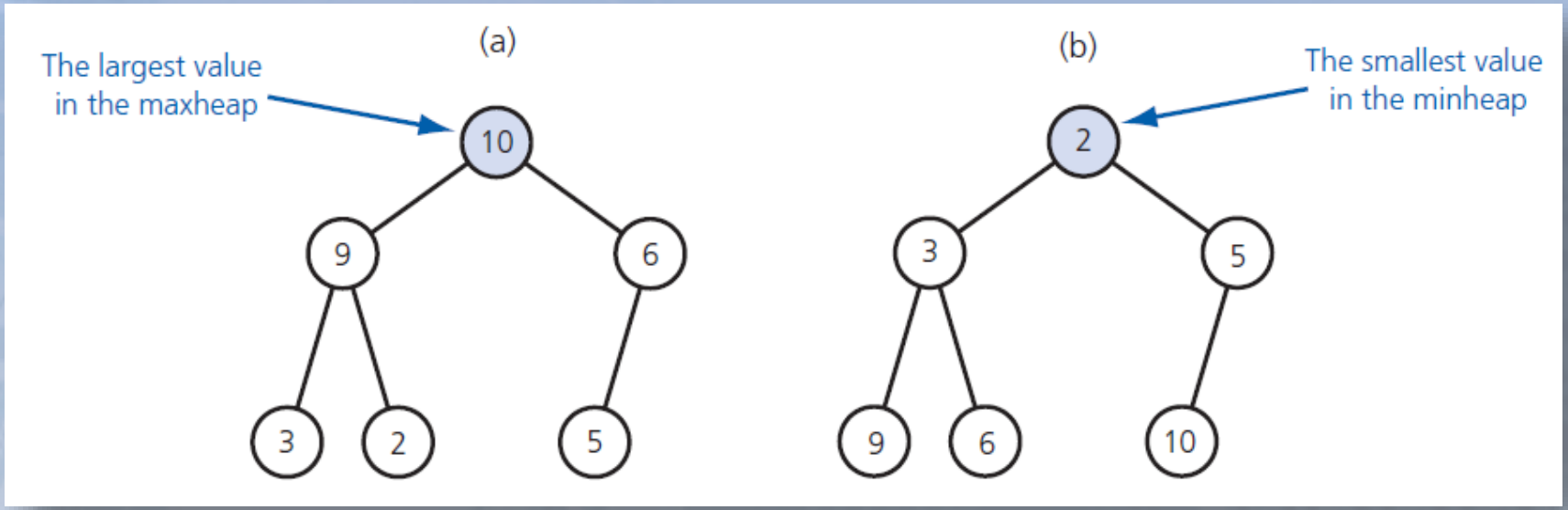
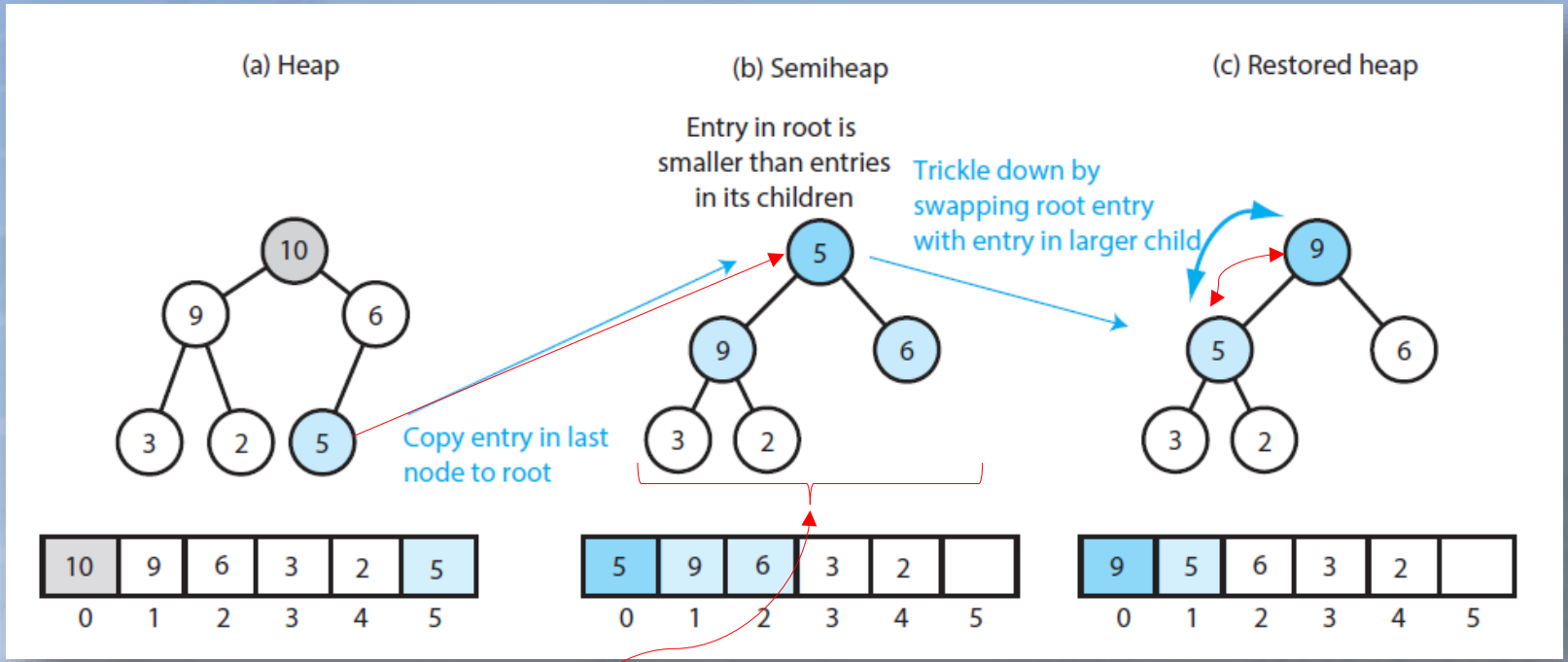


FIGURE 17-1 (a) A maxheap and (b) a minheap

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Algorithms for Array-Based Heap Operations



Remove root item (10) from a heap.

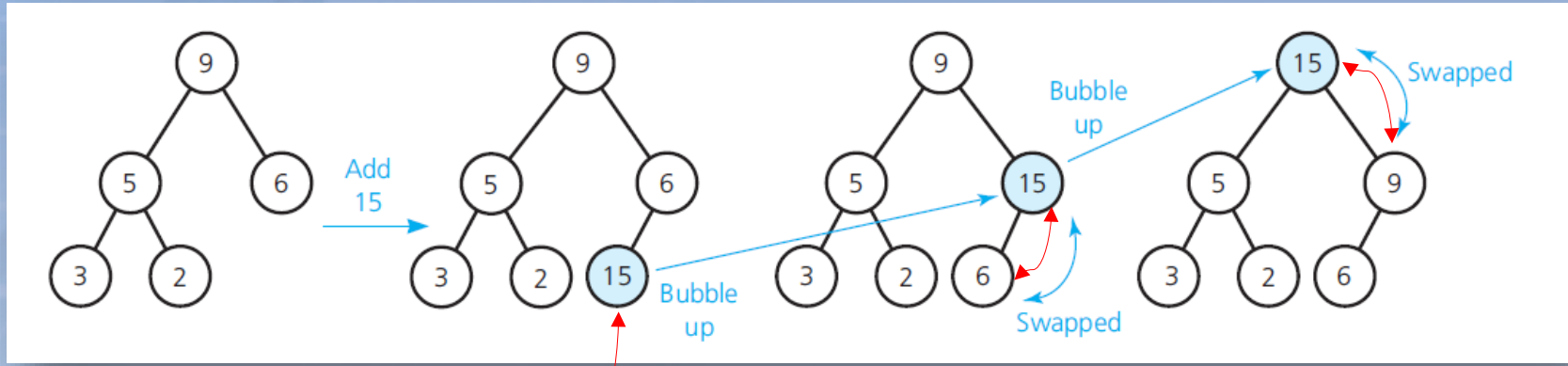
1. remove the last node of the tree and place its item in the root
2. If it is not a heap, then it is a semiheap. Recursively, trickle down the item in the root until it reaches a node in which it will not be out of place (heapify)

Removal efficiency.

- This item travels down a single path from the root to, at worst, a leaf.
- The number of array items it must swap is no greater than the height of the tree.
- Thus, remove is O(log n), which is in fact quite efficient.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Algorithms for Array-Based Heap Operations



Add an item (15) to a heap (opposite of remove).

1. Insert new item at the bottom of the tree
2. Recursively trickle up to its proper place (the parent of the node in items[i] is always stored in items[(i - 1) / 2] —unless, of course, the node is the root).

Add efficiency.

- This item travels down a single path from a leaf to the root, at worst.
- The number of array items it must swap is no greater than the height of the tree.
- Thus, add is O(log n), which is in fact quite efficient.

FIGURE 17-5 Adding 15 to a heap

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Transforming Binary Tree to Heap

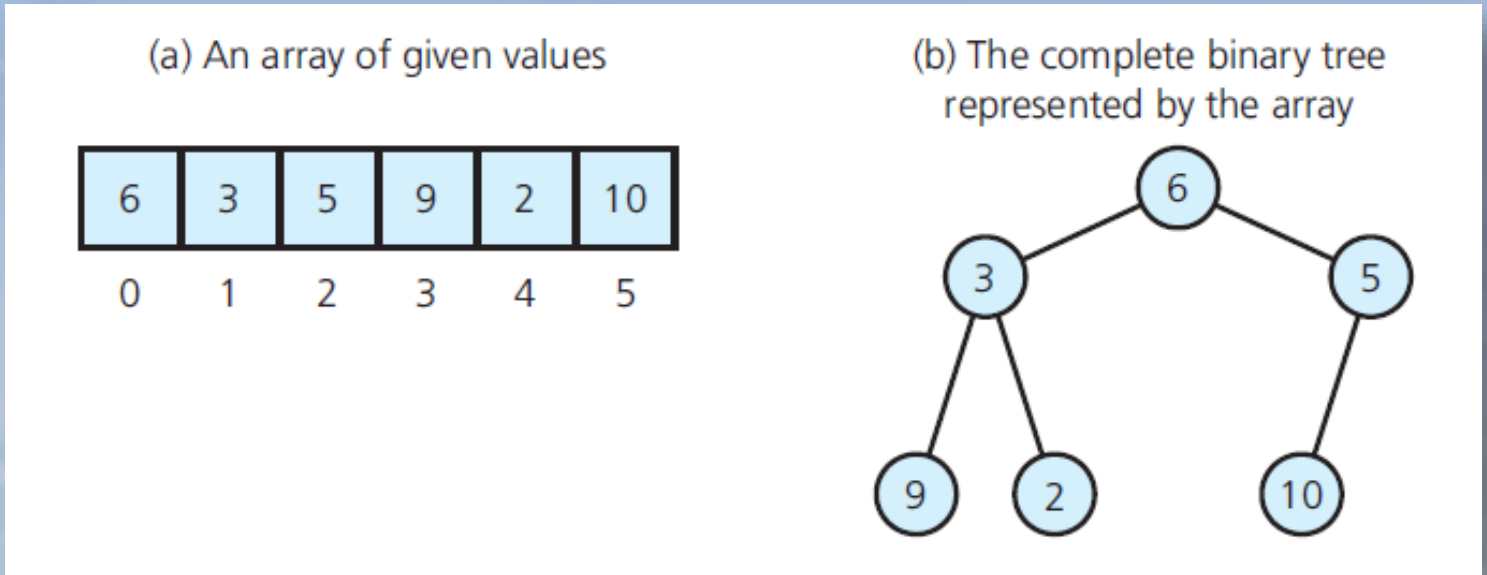
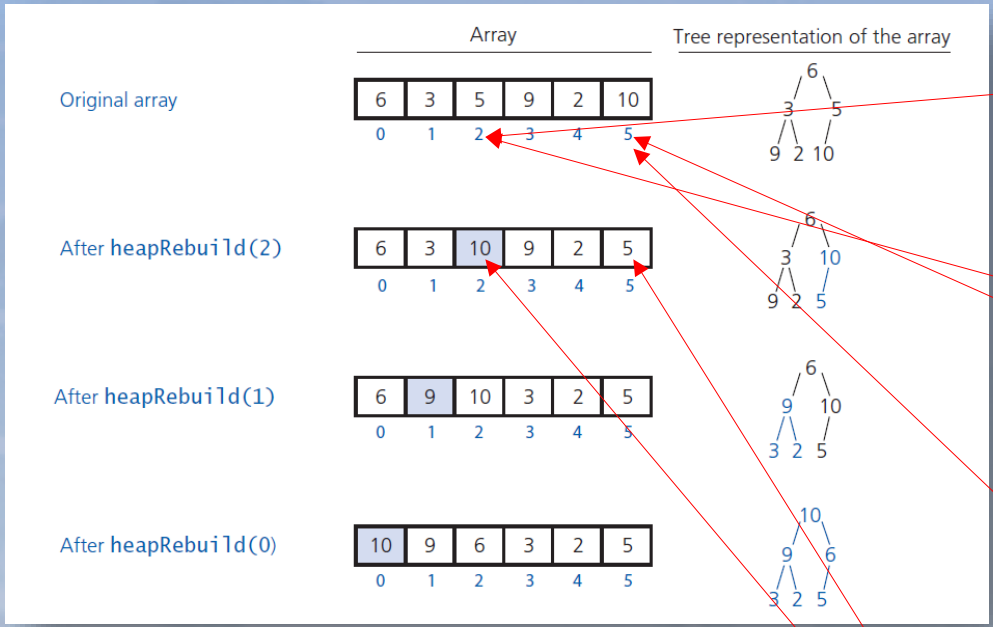


FIGURE 17-8 Array and its corresponding complete binary tree

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Transforming Binary Tree to Heap

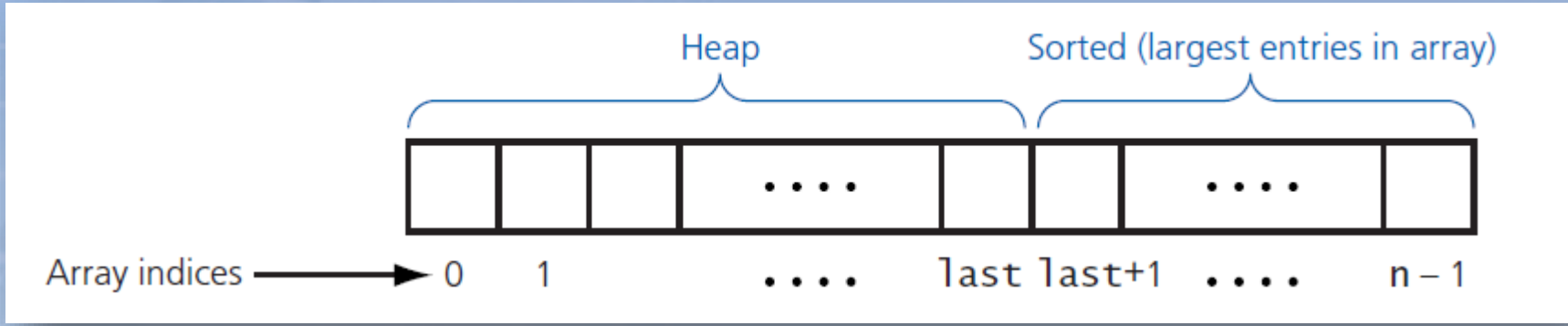



```
function buildMaxHeap(array):
    n = length of array

    # Step 1: Start from the last non-leaf node and move upwards
    for i from n/2 - 1 down to 0:  # from index 2, 1, 0
        heapRebuild(array, n, i)

function heapRebuild(array, n, i):
    largest = i    # initialize largest to root
    leftChild = 2 * i + 1  # index 5, array[leftChild] = 10
    rightChild = 2 * i + 2  # none

    # Step 2: Check if the left child exists and is greater than the
current largest
    if leftChild < n and array[leftChild] > array[largest]:
        largest = leftChild

    # Step 3: Check if the right child exists and is greater than the
current largest
    if rightChild < n and array[rightChild] > array[largest]:
        largest = rightChild

    # Step 4: If largest is not the current node, swap and
heapRebuild the affected subtree
    if largest != i:
        swap(array[i], array[largest])
        heapRebuild(array, n, largest)
```

FIGURE 17-9 Transforming an array into a heap

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Heap Sort



heap sort algorithm

1. Transform array into heap
2. For each item from heap
   a. Swap the root (largest item) with last item in the heap
      Swap(myArray[0], myArray[last])
   b. Transform heap region to heap
   c. Decrement last /* reduce heap region/increase sorted region */

FIGURE 17-10 Heap sort partitions an array into two regions

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

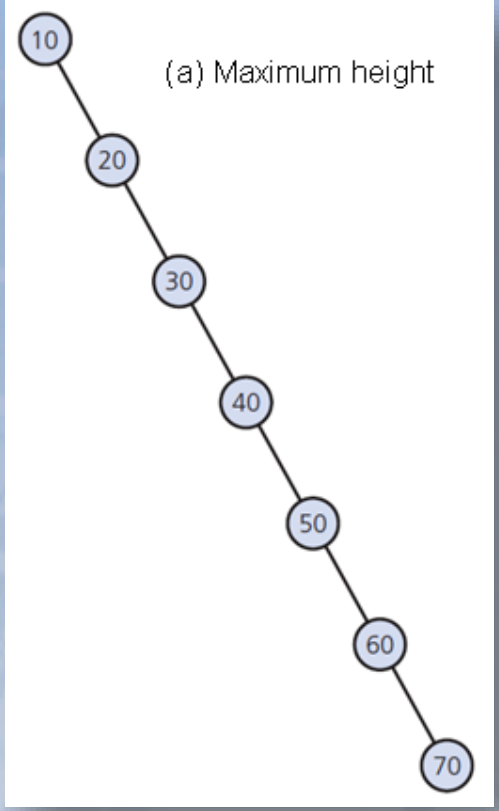
Heap Sort

- Heap sort converts an array into a heap to locate the array's largest item. This step enables the heap sort to sort an array in an efficient manner.
- Heap sort, like merge sort, has good worst-case and average-case behaviors
  - O(n * log n) // applies to the worst and average cases
- Heap sort has an advantage over merge sort in that it does not require a second array.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Balanced Search Trees

If you inserted the items: 10, 20, 30, 40, 50, 60, and 70 into the tree in ascending order, you would obtain a binary search tree of maximum height



If you inserted the items: 40, 20, 60, 10, 30, 50, and 70 into the tree in that order, you would obtain a binary search tree of minimum height

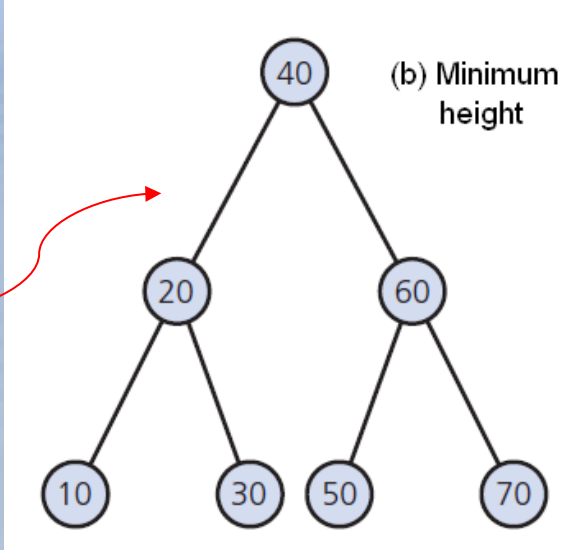


FIGURE 19-1 The tallest and shortest binary search trees containing the same data

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

AVL Trees

- An AVL tree
  - A balanced binary search tree
- Maintains its height close to the minimum
- Rotations restore the balance

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

AVL Trees – Balance Factor

- In a binary tree the **balance factor** of a node X is defined to be the height difference of its two child sub-trees rooted by node X.
- The **subtree heights are stored at each node** for all nodes in an AVL Tree, and the **balance factor is calculated based on its subtree heights to check if the tree has become out of balance**.
- The height of a subtree is the number of edges between the root node of the subtree and the leaf node farthest down in that subtree.
- Balance factor values
  - 0: The node is in balance.
  - more than 0: The node is "right heavy".
  - less than 0: The node is "left heavy".
- **If the balance factor is less than -1, or more than 1**, for one or more nodes in the tree, the tree is considered not in balance, and a rotation operation is needed to restore balance.

https://en.wikipedia.org/wiki/AVL_tree
https://www.w3schools.com/dsa/dsa_data_avltrees.php

2-3 Trees

- A 2-3 tree is a tree in which each internal node (nonleaf) has either two or three children
- All leaves are at the same level; the tree is height balanced
- Data is kept in sorted order, with left subtree containing smaller values and right subtree containing larger values
- A node with two children is called a 2-node
- A node with three children is called a 3-node

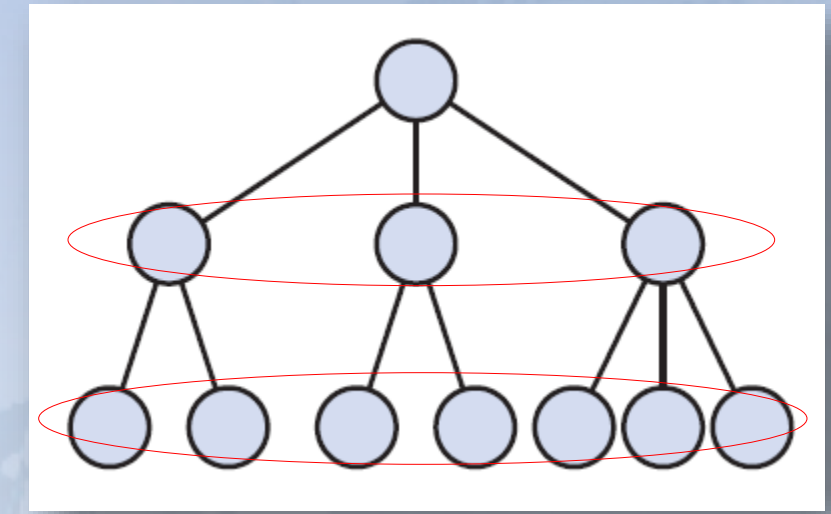


FIGURE 19-6 A 2-3 tree of height 3

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

2-3 Trees

- A 2-node, which has two (2) children, must contain a single data item that is > than the left child's item and < the right child's item
- A 3-node, which has three (3) children must contain two (2) data items, S and L, that satisfy the following relationship: S is > the left child's item(s) and < than the middle child's item(s); L > than the middle child's item(s) and < the right child's item(s)
- A leaf may contain either one (1) or two (2) data items.

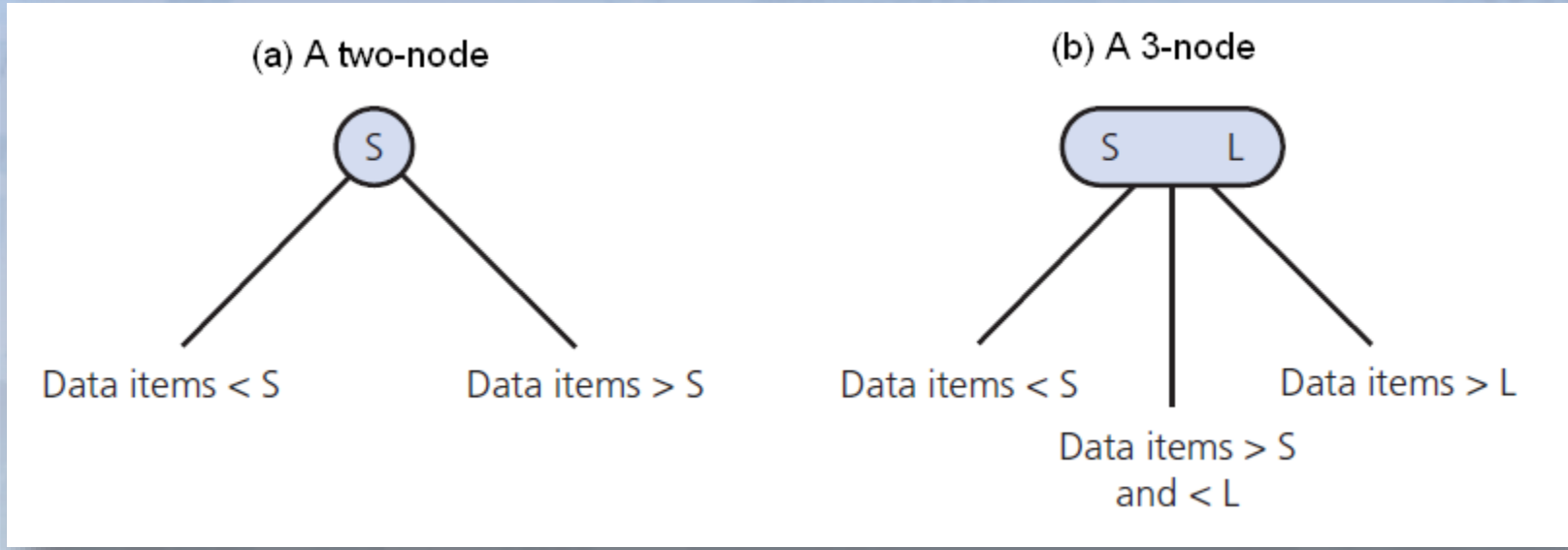


FIGURE 19-7 Nodes in a 2-3 tree

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Searching a 2-3 Tree

- Search of a 2-3 and shortest binary search tree approximately same efficiency
  - A binary search tree with *n* nodes cannot be shorter than $\log_2(n + 1)$
  - A 2-3 tree with n nodes cannot be taller than $\log_2(n + 1)$
  - Node in a 2-3 tree has at most two data items
  - Although you might visit fewer nodes when searching a 2-3 tree (vs a binary tree), you might have to make more comparisons at each node
- Searching 2-3 tree is $O(\log n)$

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Searching a 2-3 Tree

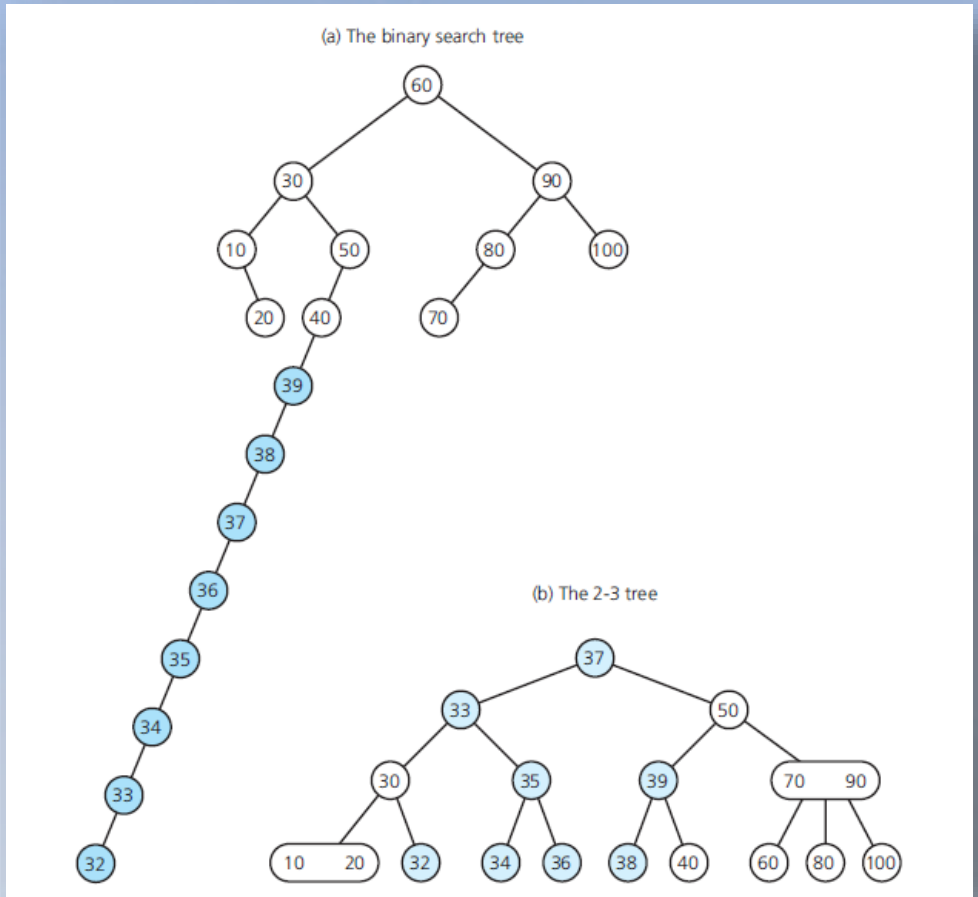


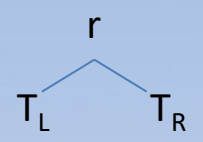
FIGURE 19-10 The trees of Figure 19-9 after adding the values 39 down to 32

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

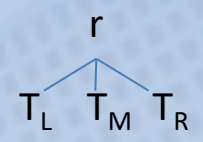
2-3-4 Trees

T is a 2-3-4 tree of height h if one of the following is true:

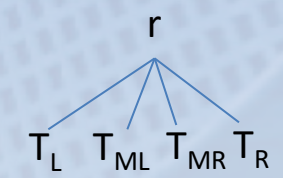
1. T is empty, in which case h is 0.
2. T is of the form
   a. r is a node that contains one data item
   b. T_L and T_R are both 2-3-4 trees, each of height h-1
   c. $r > T_L$ and $r < T_R$



3. T is of the form
   a. r is a node that contains two data items;
   b. T_L, T_M and T_R are both 2-3-4 trees, each of height h-1
   c. smaller $r > T_L$ and $r < T_M$; larger $r > T_M$ and $< T_R$



4. T is of the form
   a. r is a node that contains three data items;
   b. T_L, T_{ML}, T_{MR} and T_R are both 2-3-4 trees, each of height h-1
   c. smallest $r > T_L$ and smallest $r < T_{ML}$
   d. middle $r > T_{ML}$ and middle $r < T_{MR}$
   e. largest $r > T_{MR}$ and largest $r < T_R$



© 2017 Pearson Education, Hoboken, NJ. All rights reserved

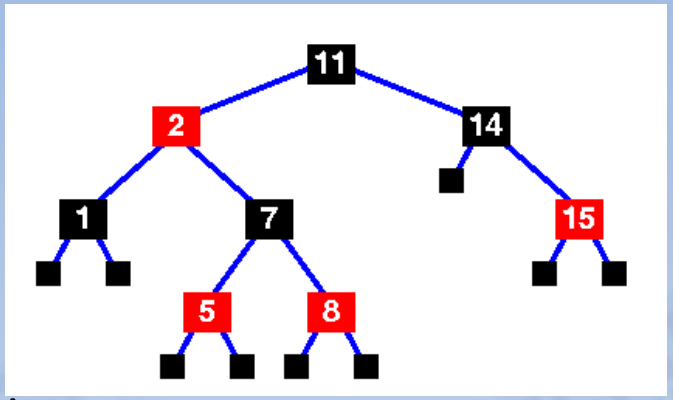
2-3-4 Trees

- Searching and traversing
  - Simple extensions of corresponding algorithms for a 2-3 tree
- Adding data
  - Like addition algorithm for 2-3 tree
  - Splits node by moving one data item up to parent node

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Red-Black Trees

- [1]Invented in 1978 Leonidas Guibas and Robert Sedgewick
- **Self balancing binary search tree**; O(log n) search/add/delete
- [2]Red-black tree properties
  - **Each node is either red or black**
  - Root node is always black in color
  - Maintain balance during add/remove operations
  - Nodes that have no child are considered the internal nodes and these nodes are connected to the NIL nodes that are always black in color. **The NIL nodes are the leaf nodes in the Red-Black tree**.
  - **If the node is Red, then its children should be in Black color**.
  - Every path from a node to any of its descendant's NIL node should have same number of black nodes.



[3]A basic red-black tree

[1]https://en.wikipedia.org/wiki/Red%E2%80%93black_tree#:~:text=AVL%20trees%20can%20be%20colored,trees%20are%20more%20rigidly%20balanced.
[2]https://www.javatpoint.com/red-black-tree
[3]https://www.eecs.umich.edu/courses/eecs380/ALG/red_black.html

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

2-3-4 vs Red-Black Trees

- A 2-3-4 tree:
  - Balanced
  - Add/remove operations use one pass from root to leaf
  - Requires more storage than binary search tree
- Red-black tree
  - Special binary tree which represents 2-3-4 tree
  - Advantages of a 2-3-4 tree but requires less storage
- Represent each 3 node and 4 node in a 2-3-4 tree as equivalent binary search tree
  - All child pointers in original 2-3-4 tree are black pointers
  - Red pointers link 2-nodes that now contain values that were in a 3-node or a 4-node.
- Worst case search, insert and delete O(log n)

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Searching and Traversing a Red-Black Tree

- A red-black tree is a binary search tree
- Thus, search and traversal
  - Use algorithms for binary search tree

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Red-Black vs AVL Trees

	Red-Black	AVL
[1, 2]Height	2 * log (n + 1)	1.44 * log n
[3]Add/delete operations	faster add/delete operations; fewer rotations are due to relatively balancing	complex add/delete operations; more rotations are due to strict balancing
[3]Storage	requires only 1 bit of information per node	store balance factors or heights with each node thus requiring storage for an integer per node
[3]Uses	used in most of the language libraries like map, multimap, multiset in C++, etc	used in databases where faster retrievals are required
[3]Balancing	Take less processing for balancing i.e.; maximum two rotation required	Take more processing for balancing

[1]https://pages.cs.wisc.edu/~siff/CS367/Notes/rbts.html#:~:text=It%20can%20be%20proven%20that,O(lg(n)).
[2]https://pages.cs.wisc.edu/~ealexand/cs367/NOTES/AVL-Trees/index.html
[3]https://www.geeksforgeeks.org/red-black-tree-vs-avl-tree/

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

B-Tree

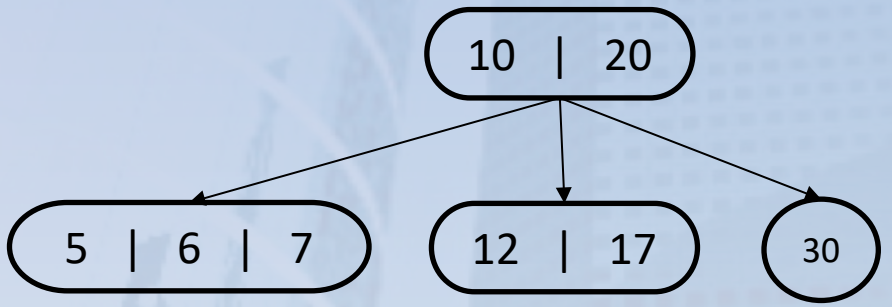
- A self-balancing multiway search tree
- Designed for efficient data access in large datasets
- Common in:
  - Databases
  - File systems
  - Indexing structures

B-Tree

- Generalization of 2-3 or 2-3-4 trees
- Every node stores multiple keys
- Nodes may have multiple children (not just two)
- All leaves are at the same depth
- Keeps height low, even for large amounts of data
- Balanced structure with sorted keys
- Keeps disk access minimal

B-Tree

- Maximum children per node: m
- Maximum keys per node: m – 1
- Minimum children per node: ⌈m/2⌉
- Minimum keys per node: ⌈m/2⌉ - 1
- Root can have fewer keys/children



B-tree of order 4

B-Tree

System / Context	Typical Order (m)
Small in-memory B-trees	4–10
File systems (e.g., NTFS, ext4)	100–200
Databases (MySQL, PostgreSQL, Oracle)	100–1000+
B+ trees in modern SSD-optimized systems	128–512+

B-Tree

- Search: O(log n)
- Insert/Delete: O(log n)
- Few levels = faster disk access
- Especially efficient when disk I/O is expensive
- Designed for external storage
- Minimize disk reads/writes
- Store many keys per node → shallow trees
- Great for:
  - Database indexes (e.g., MySQL, PostgreSQL)
  - File systems (e.g., NTFS, HFS+, ext4)

End